

PREDICTING PITCH SEQUENCES IN BASEBALL USING MULTINOMIAL LOGISTIC REGRESSION

A Thesis Submitted in Partial Fulfillment
of the Requirements for the Degree of
Master of Science

by

Brian Bruxvoort

Dr. Scott Alberts, Thesis Advisor
Department of Computer and Data Sciences
School of Science and Mathematics
2026

TRUMAN STATE UNIVERSITY
Kirksville, Missouri

ACKNOWLEDGEMENTS

First and foremost, I would like to thank my amazing family, who has supported me every step of the way. This thesis is not just the culmination of my academic journey - it is a shared accomplishment that I could not have achieved without you. Being surrounded by so much love and encouragement has given me the confidence to know that anything is possible.

I would also like to extend my sincere gratitude to my thesis advisor and committee members. Having worked alongside many of you throughout my graduate studies, your guidance, support, and investment in my success have not gone unnoticed and is deeply appreciated. I am especially grateful for your willingness to provide thoughtful feedback and constructive critiques, which were invaluable in helping me produce a finished product I am proud of.

I would like to offer a special thank you to Dr. Alberts, who has been instrumental in my success since the very first day I stepped on campus as a transfer undergraduate student. Transfer students can often feel like outsiders when entering a new program, and your immediate warmth and encouragement made all the difference. Thank you for welcoming me so wholeheartedly into the statistics and data science community.

I would also like to thank Dr. DeCock, who served as an incredible mentor to me beyond the classroom. Your guidance around career goals and aspirations helped shape the direction of my path in ways I will always be grateful for. Most importantly, your encouragement to continue my education and pursue a Master's degree was the push I needed, and this thesis stands as a direct result of that conversation.

To Camden, Bryce, and Evan, thank you for being such fantastic people to work with during my undergraduate years. Your friendship made that chapter of my academic journey truly memorable, and I am lucky to have worked with you during that time.

I would also like to acknowledge my coaches and teammates throughout my baseball career. The sport taught me so much including discipline, perseverance, resiliency and the value of being part of something bigger than yourself. The passion for the game that you all helped foster in me extends far beyond the field, and your influence played a meaningful role in getting me to where I am today.

I would also like mention Rob Friedman, also known as “Pitching Ninja,” whose YouTube videos and in-depth analysis of pitching ideology shaped my development as a pitcher. His work was also a direct source of inspiration for this thesis, as it sparked the idea of blending my background in data science with my lived experience as a collegiate baseball pitcher. That intersection of passion and education is what this research is truly built on.

Finally, I want to reflect on just how much this journey has meant to me. What began as a love of baseball and a curiosity for numbers has grown into something I could have never imagined. This thesis represents not only years of hard work and learning, but the coming together of every mentor, teammate, friend, and family member who believed in me along the way. I am deeply grateful for all of it, and I carry each of your influences with me as I wrap up my educational career.

TABLE OF CONTENTS

LIST OF TABLES	v
LIST OF FIGURES	vi
1. Introduction	1
1.1. Research Objectives	2
1.2. Literature Review	3
1.2.1 The Strategic Role of Pitch Sequences	3
1.2.2 Machine Learning Approaches to Pitch Prediction	4
1.2.3 Logistic Regression Models for Pitch Prediction	5
1.2.4 Research Gaps and Present Study	6
2. Data	7
2.1. Variable Selection	7
2.1.1 Load Required Libraries	9
2.1.2 Load Raw Data	9
2.2. Descriptive Statistics	9
2.2.1 Pitch Type Distribution	10
2.2.2 Zone Distribution	12
2.2.3 Count Distribution	14
2.2.4 Batter Handedness	15
2.2.5 Pitch Sequences	16
2.2.6 Pitch Type by Zone	17
3. Methodology	19
3.1. Research Design	19
3.2. Theoretical Framework	20
3.2.1 Multinomial Logistic Regression Framework	20
3.2.2 Methodological Innovation: Lagged Variables	20
3.2.3 Alternative Approaches Considered	21
3.3. Model Specifications	22
3.3.1 Model 1: Simple Model	22
3.3.2 Model 2: Simple Model with Batter Handedness	23
3.3.3 Model 3: Basic Model with Count and Zone	23
3.3.4 Model 4: Separate Type and Zone Models	24
3.3.5 Model 5: Interaction Model	24
3.4. Model Training and Evaluation	25
3.4.1 Data Preparation	25

3.4.2	Model Fitting Process	26
3.4.3	Performance Evaluation	26
3.5.	Statistical Comparison of Models	26
3.5.1	Pairwise Comparisons	27
3.5.2	Analysis of Variance	27
3.5.3	Effect Size Calculation	27
3.6.	Computational Implementation	27
4.	Results	29
4.1.	Overall Model Performance	29
4.1.1	Summary Statistics	29
4.1.2	Visualization of Model Performance	32
4.2.	Statistical Significance Testing	35
4.2.1	Analysis of Variance Results	37
4.2.2	Tukey's HSD Post-Hoc Comparisons	38
4.2.3	Effect Size Analysis	39
4.3.	Pitcher-Specific Performance	40
4.4.	Model Complexity vs. Performance	45
5.	Discussion	47
5.1.	Key Findings Summary	47
5.2.	Interpretation	49
5.2.1	Why Model 3 Dominates	49
5.2.2	Why Simple Models Perform Adequately	49
5.2.3	Why the Interaction Model Failed	50
5.2.4	Pitcher-Specific Patterns	51
5.3.	Practical Applications	52
5.3.1	Selective Implementation by Opponent	52
5.3.2	Pre-Game Scouting Enhancement	52
5.3.3	Pitcher Self-Analysis and Development	53
5.4.	Limitations	53
5.4.1	Data and Temporal Limitations	53
5.4.2	Methodological Limitations	54
5.4.3	Contextual Limitations	54
5.5.	Future Research Directions	55
5.5.1	Multi-Season Temporal Analysis	55
5.5.2	Hierarchical and Bayesian Modeling	55
5.5.3	Alternative Modeling Frameworks	56
5.5.4	Substantive Baseball Research Questions	56
6.	Conclusion	57
7.	R Shiny App Link	59
	References	60
	Appendix - R Code	61
7.1.	Appendix A: Model Building Code	61

7.1.1	Model 1: Simple Model	61
7.1.2	Model 2: Simple Model with Batter Handness (Stand)	66
7.1.3	Model 3: Basic Model	71
7.1.4	Model 4: Separate Type and Zone Models	76
7.1.5	Model 5: Interaction Model	85

LIST OF TABLES

4.1	Overall Model Performance Summary	29
4.2	Top 10 Most Predictable Pitchers (Highest Mean Accuracy)	40
4.3	Top 10 Least Predictable Pitchers (Lowest Mean Accuracy)	41

LIST OF FIGURES

2.1	Distribution of Pitch Types in 2025 MLB Season	10
2.2	Distribution of Pitch Locations Across Strike Zones	12
2.3	Strike Zone Layout with 14 Zones	13
2.4	Distribution of Ball-Strike Counts	14
2.5	Batter Handedness Distribution	15
2.6	Top 15 Most Common Two-Pitch Sequences	16
2.7	Top 15 Most Common Pitch Type-Zone Combinations	17
4.1	Model Accuracy Comparison Across All Pitchers	32
4.2	Accuracy Distribution by Model	34
4.3	Distribution of Mean Accuracy Across Pitchers	43
4.4	Model Efficiency - Accuracy per Complexity Unit	45

PREDICTING PITCH SEQUENCES IN BASEBALL USING MULTINOMIAL LOGISTIC REGRESSION

An Abstract of the Thesis by
Brian Bruxvoort

This research develops multinomial logistic regression models for predicting pitch sequences in Major League Baseball using Statcast data from the 2025 regular season. Individual models were constructed for each pitcher rather than a single league-wide model, with predictors restricted to observable game state variables including pitch history, ball-strike count, batter handedness, and pitch location zone. The comprehensive model achieved 62.36% classification accuracy, substantially exceeding the 32% naive baseline. Pitcher predictability varied considerably, with relief pitchers proving systematically more predictable than starting pitchers due to their narrower repertoires and more defined roles. A joint pitch type and location model failed due to severe data sparsity created by the expansion of outcome categories. These findings offer practical implications for scouting and hitter preparation while demonstrating that classical statistical methods can effectively model sequential pitch selection patterns.

1. Introduction

Baseball, often referred to as America's pastime, has evolved from a simple game of bats and balls into a complex strategic battle where every pitch matters. At the heart of this battle lies one of the game's most intriguing questions: what pitch will the pitcher throw next? This seemingly simple question has been the focus of players, coaches, and analysts for generations, as the ability to anticipate the next pitch can dramatically shift the competitive balance between hitter and pitcher.

A hitter who successfully anticipates the type and location of an incoming pitch gains precious milliseconds to react which is a critical advantage in a sport where a 95 mph fastball travels from the pitcher's hand to home plate in approximately 400 milliseconds (Boddy, 2012). This competitive advantage has led to numerous controversies throughout baseball history, from traditional sign-stealing techniques to more recent scandals involving electronic devices, as exemplified by the 2017 Boston Red Sox incident where the team allegedly used Apple Watches to decode opposing catchers' signals (Stratton, 2018).

As technology has advanced, advanced analytics and machine learning have been introduced. Due to this advancement, the approach to understanding and predicting pitch sequences has fundamentally transformed. Modern baseball generates vast quantities of data through systems like MLB's Statcast, which tracks every pitch thrown in every game with unprecedented detail (Major League Baseball, 2024). This data revolution has created new opportunities for researchers to apply sophisticated statistical and computational methods to the age-old problem of pitch prediction.

This thesis addresses the challenge of predicting pitch sequences in Major League Baseball

using multinomial logistic regression models. Unlike previous research that has focused primarily on neural networks or support vector machines, this study leverages the interpretability and efficiency of logistic regression while building individual models for each pitcher in the 2025 MLB season. By focusing on readily observable game state variables which include count, batter handedness, and pitch history, this research aims to create a practical, implementable system that could be utilized in real-time during games.

1.1 Research Objectives

The primary objective of this research is to develop and evaluate multiple iterations of multinomial logistic regression models that predict pitch type based on easily trackable game situation variables. Rather than pursuing the absolute highest possible prediction accuracy through complex feature engineering, this study prioritizes practical usability and real-world applicability.

This research makes several contributions to the existing literature on pitch prediction. First, it provides a systematic comparison of multinomial logistic regression models with varying levels of complexity, demonstrating the tradeoffs between model sophistication and prediction accuracy. Second, it builds separate models for each pitcher rather than a single league-wide model, accounting for individual pitching styles and tendencies. Third, it emphasizes practical implementation by restricting predictors to variables that are readily observable during live game situations. Finally, it provides comprehensive statistical analysis comparing model performance across different specifications, offering insights into which game state variables contribute most significantly to pitch prediction accuracy.

1.2 Literature Review

The study of pitch sequences and pitch prediction in baseball has evolved significantly over the past two decades, driven by advances in data collection technology and analytical methodologies. This literature review examines three interconnected areas of research: the strategic importance of pitch sequences, various approaches to predicting pitch types, and the specific application of logistic regression models to this problem.

1.2.1 The Strategic Role of Pitch Sequences

Clay (2023) conducted one of the most comprehensive recent studies on two-pitch sequence behavior using discrete-time Markov chains to analyze pitch location patterns. Examining data from 87 American League pitchers during the 2019 season, Clay found that pitchers could be grouped into five distinct clusters based on their two-pitch location sequence behaviors, with significant differences in performance metrics between groups. Notably, the highest-performing group included elite pitchers like Justin Verlander and Gerrit Cole, who demonstrated what Clay termed “situational variance” which as he put it, the elite pitchers varied their two-pitch sequences more than other groups in certain game situations while remaining consistent in others, particularly when beginning and ending at-bats.

Boddy (2012) provided practical insights into pitch sequencing strategies from a training perspective, emphasizing data driven decision making in pitch selection. Drawing on large scale data mining of MLB pitches, Boddy outlined how different pitch types serve distinct strategic purposes. Fastballs establish velocity expectations and generate strikes, curveballs disrupt timing,

sliders generate strikeouts, and change-ups/sinkers induce groundballs. His analysis revealed that sliders have the highest “whiff rate” while curveballs have the highest “watch rate,” suggesting that pitch effectiveness depends heavily on game situation and previous pitch selection (Boddy, 2012).

1.2.2 Machine Learning Approaches to Pitch Prediction

The application of machine learning to pitch prediction has expanded dramatically since the early 2010s, with researchers using increasingly sophisticated techniques. Ganeshapillai and Guttag (2012) conducted work in this area as they developed linear support vector machines to predict pitch types for individual pitchers. Using data from the 2008 and 2009 MLB seasons, they built separate models for 359 pitchers, incorporating 48 features including previous pitch, count, score differential, base runners, and pitcher tendencies. Their analysis revealed that previous pitcher actions specific to the count and pitcher behavior toward specific batters were among the most important predictive features (Ganeshapillai & Guttag, 2012).

More recent work has leveraged neural networks and deep learning architectures. Augustine (2024) developed “MLB Pitch Predictor” using Azure Machine Learning Studio, applying a Multiclass Boosted Decision Tree algorithm to predict pitches thrown by Tyler Glasnow. After incorporating data from both 2023 and early 2024 seasons, Augustine achieved a 66% overall accuracy rate, with the model reaching 73% accuracy in specific game applications. Importantly, Augustine’s work highlighted practical limitations of pitch prediction models, including a pitcher’s ability to adjust strategies mid-game, park factors, umpire strike zone variability, and the fundamental unpredictability of human decision-making (Augustine, 2024).

Lee (2022) expanded the prediction problem beyond pitch type to simultaneously predict

both pitch type and location, classifying pitches into 34 categories. Using an ensemble of deep neural networks, Lee achieved 62.4% accuracy compared to a naive baseline of 8.2%. Interestingly, Lee’s analysis revealed that prediction accuracy increased in specific game situations, particularly when the pitcher’s team was losing, when the count was unfavorable to the pitcher, or when a runner was in scoring position (Lee, 2022). These findings suggest that pitchers become more predictable under pressure, following established patterns more closely when game situations are disadvantageous.

1.2.3 Logistic Regression Models for Pitch Prediction

Despite the expansion of complex machine learning techniques, logistic regression remains relevant for pitch prediction due to its interpretability, computational efficiency, and strong performance when appropriately specified. Stratton (2018) conducted extensive research on pitch prediction using Bayesian hierarchical multinomial regression, specifically analyzing Clayton Kershaw’s pitching data from 2008 to 2014. Stratton’s approach grouped pitches by catcher, recognizing that different catcher-pitcher relationships might result in different pitch calling patterns (Stratton, 2018).

Stratton employed a Block Metropolis-within-Gibbs sampler to estimate the hierarchical model, that predicted pitch type based on count and previous pitch type. A key finding was that the hierarchical model significantly outperformed its non-hierarchical counterpart, providing “extremely strong evidence” that accounting for catcher-specific effects improved prediction quality. The hierarchical modeling approach offers particular advantages through shrinkage estimators that “borrow information” across groups in the hierarchy, resulting in more stable estimates for catchers

with limited data (@ Stratton, 2018).

Hastie and Powers (2011) explored the interpretability advantages of logistic regression in predicting baseball outcomes, emphasizing how regression coefficients can be directly interpreted as log-odds ratios. This interpretability allows coaches and analysts to understand not just which pitch is most likely, but why that information may be as valuable as the prediction itself (Hastie & Powers, 2011).

1.2.4 Research Gaps and Present Study

While existing research has made substantial progress in understanding and predicting pitch sequences, several gaps remain. First, much of the recent research has focused on maximizing prediction accuracy through complex neural networks, sometimes at the expense of practical implementability and interpretability. Second, many studies have built league-wide models rather than pitcher-specific models, potentially missing important individual differences in pitching strategy. Third, few studies have systematically compared models of increasing complexity to identify the optimal balance between sophistication and performance.

This thesis addresses these gaps in a few ways. The first is developing multinomial logistic regression models that prioritize practical usability while maintaining competitive accuracy. Second, it builds separate models for each pitcher to account for individual tendencies and styles. Third, it systematically compares five model specifications of increasing complexity to identify which additional features provide meaningful improvements in prediction accuracy. Lastly, it uses only variables that are readily observable during live games, ensuring the models could be implemented in real-time situations.

2. Data

The foundation of this research is Statcast data from the 2025 Major League Baseball regular season, obtained through the MLB's BaseballSavant website. Statcast is MLB's official tracking technology, using high-resolution cameras and radar equipment to record detailed information about every pitch, swing, and play across all MLB games.

Due to BaseballSavant's limitations on quantity of exportable pitch data, the pitch data for each individual team was exported individually. BaseballSavant limits the user to exporting a maximum of 25,000 rows (pitches) of data. Thankfully, each team threw no more than 25,000 pitches throughout the 2025 season. After the pitch data for each team was exported, they were then combined into one dataset using joins so that the analysis could be completed.

The newly created raw dataset contained 118 variables capturing diverse aspects of each pitch, including pitch velocity, spin rate, release point coordinates, pitch movement characteristics, batted ball outcomes, and contextual game state information. The complete dataset comprised 712,528 individual pitch observations, which represents the total number of pitches thrown across all MLB games during the 2025 regular season.

2.1 Variable Selection

For the purposes of this research, a subset of variables was selected based on two primary criteria. The first was relevance to pitch prediction modeling, and the second was practical observ-

ability during live game situations. The following variables were retained for analysis:

- **pitch_type**: The classification of pitch type (e.g., four-seam fastball, slider, curveball)
- **player_name**: The name of the pitcher
- **zone**: The pitch location zone within or around the strike zone (14 distinct zones)
- **stand**: Batter handedness (left-handed or right-handed)
- **balls**: The number of balls in the current count
- **strikes**: The number of strikes in the current count
- **at_bat_number**: Sequential identifier for the at-bat within the game

This variable selection strategy reflects a deliberate emphasis on model interpretability and practical implementation. While more sophisticated feature engineering incorporating variables such as spin axis, release height, or pitch movement characteristics could potentially enhance predictive accuracy, such an approach introduces several complications. First, these variables are not readily observable during live games without specialized equipment. Second, extensive feature engineering increases the risk of model overfitting. Third, many of the 118 available variables exhibit substantial multicollinearity. For instance, spin axis, velocity, and horizontal/vertical break are all interrelated and collectively determine pitch type.

By restricting the analysis to basic, observable game state variables, this research prioritizes the development of models that are simultaneously interpretable, computationally efficient, and implementable in real-time game contexts.

2.1.1 Load Required Libraries

2.1.2 Load Raw Data

Dataset dimensions: 712528 rows x 118 columns

2.2 Descriptive Statistics

Before getting into the modeling and analysis, exploratory analysis of the 2025 MLB pitch data was completed to examine the distribution of key variables and identifying patterns that will inform the modeling approach.

2.2.1 Pitch Type Distribution

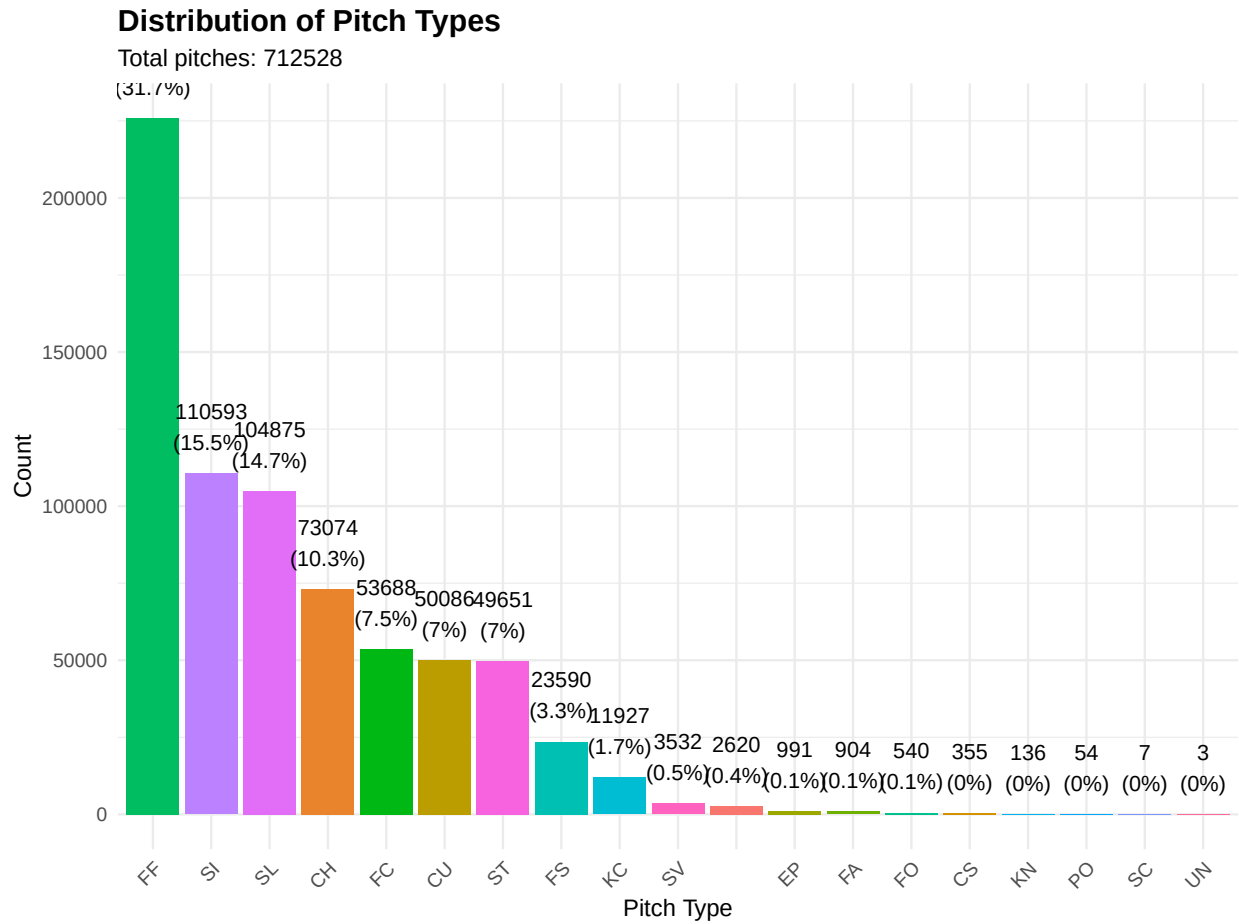


Figure 2.1: Distribution of Pitch Types in 2025 MLB Season

The pitch type abbreviations used in this paper are defined as follows:

- **CH** - Changeup
- **CU** - Curveball
- **FC** - Cutter
- **EP** - Eephus
- **FO** - Forkball

- **FF** - Four-Seam Fastball
- **KN** - Knuckleball
- **KC** - Knuckle-curve
- **SC** - Screwball
- **SI** - Sinker
- **SL** - Slider
- **SV** - Slurve
- **FS** - Splitter
- **ST** - Sweeper

Figure 2.1 displays the frequency distribution of pitch types across all 712,528 pitches in the dataset. Four-seam fastballs (FF) dominate MLB pitching, accounting for 231,059 pitches (31.7% of all pitches thrown). Sinkers (SI) represent the second-most common pitch type with 110,593 pitches (15.5%), followed by sliders (SL) at 104,875 pitches (14.7%) and changeups (CH) at 73,074 pitches (10.3%).

The dominance of fastballs/sinkers and the relative frequency of other pitch types establish a baseline expectation: any effective prediction model should at minimum perform better than a naive baseline that always predicts the most common pitch type. Such a baseline would achieve 31.7% accuracy if fastballs were the only predicted pitch, which is a benchmark that contextualizes future model performance.

2.2.2 Zone Distribution

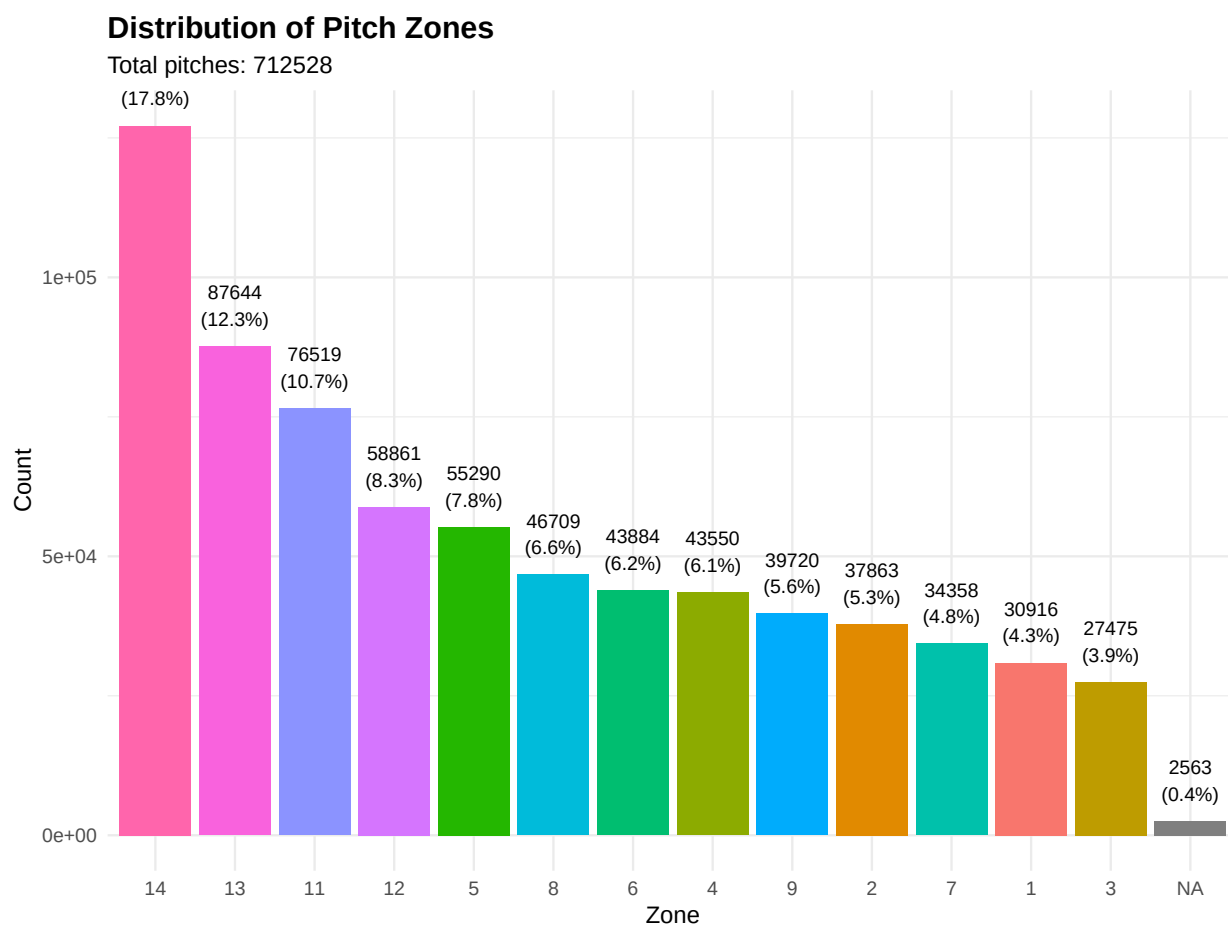


Figure 2.2: Distribution of Pitch Locations Across Strike Zones

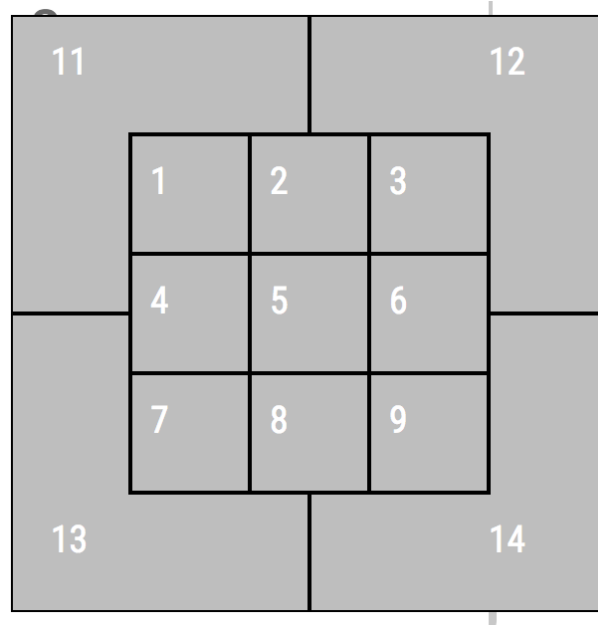


Figure 2.3: Strike Zone Layout with 14 Zones

Note: The zone graphic is from the catcher/umpire's point of view.

Figure 2.2 illustrates the spatial distribution of pitches across the 14 defined strike zones. Zone 14, representing pitches outside and off the plate, contains the highest frequency of pitches at (17.8%). The distribution reveals that approximately 60% of pitches are thrown within the official strike zone (zones 1-9), with the remaining 40% thrown outside the zone.

2.2.3 Count Distribution

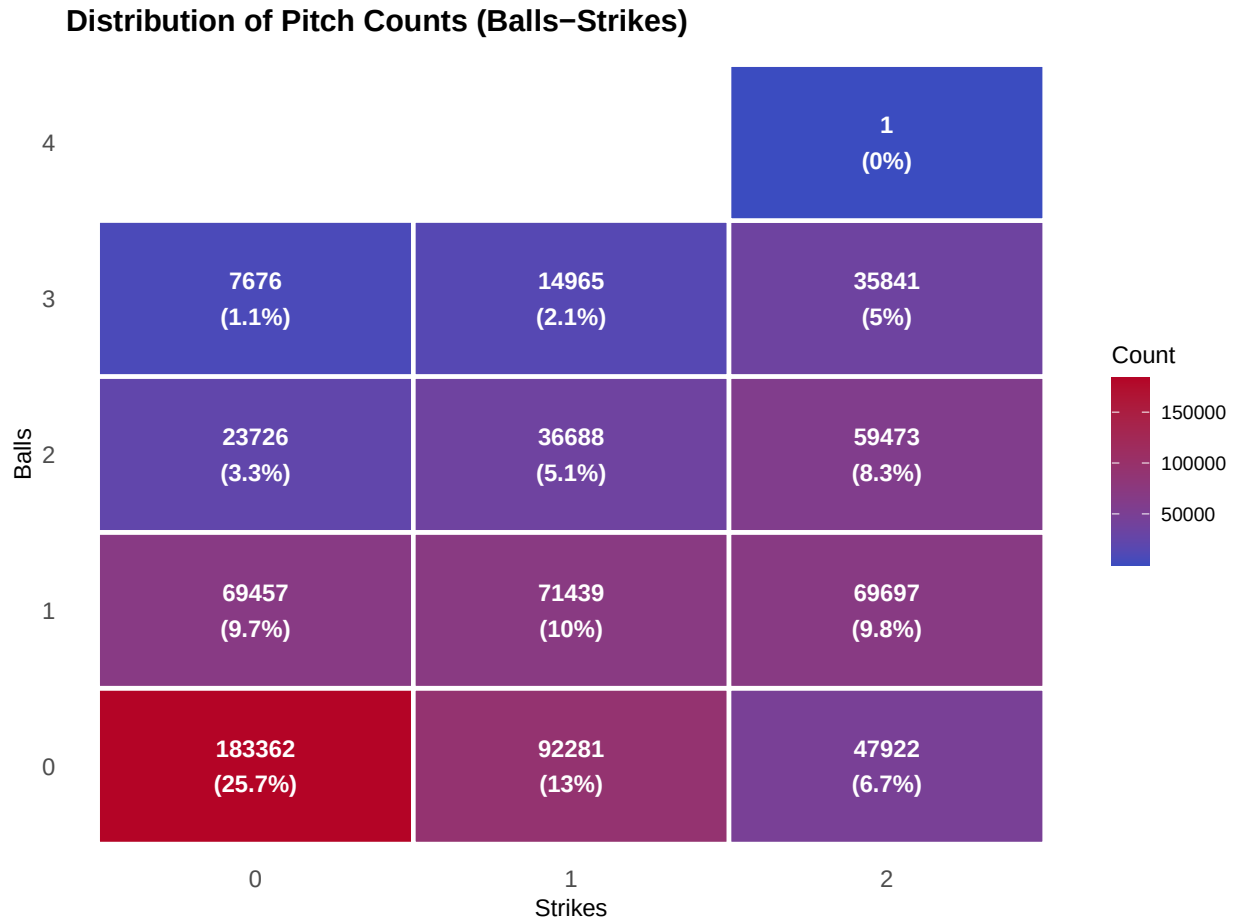


Figure 2.4: Distribution of Ball-Strike Counts

Figure 2.4 presents the frequency distribution of all possible ball-strike count combinations. The most common count is 0-0 (183,362 pitches, 25.7%), which necessarily appears at the beginning of every plate appearance. Two-strike counts collectively account for approximately 30% of all pitches, while full counts (3-2) occur with moderate frequency (35,841 pitches, 5.0%).

2.2.4 Batter Handedness

Batter Handedness Distribution (Pie Chart)

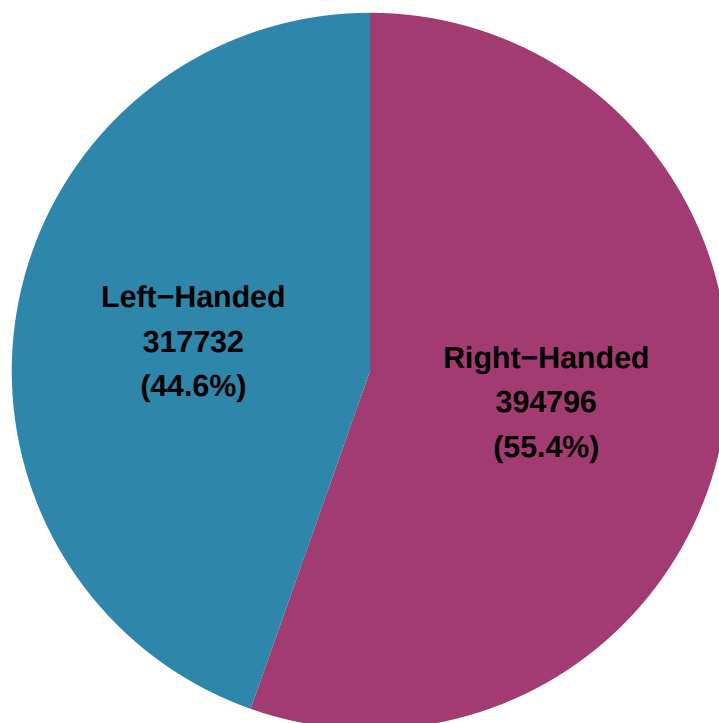


Figure 2.5: Batter Handedness Distribution

Figure 2.5 displays the distribution of batter handedness. Right-handed batters faced 394,796 pitches (55.4%), while left-handed batters faced 317,732 pitches (44.6%). This approximate 55-45 split ensures sufficient data to estimate handedness-specific effects with reasonable precision.

2.2.5 Pitch Sequences

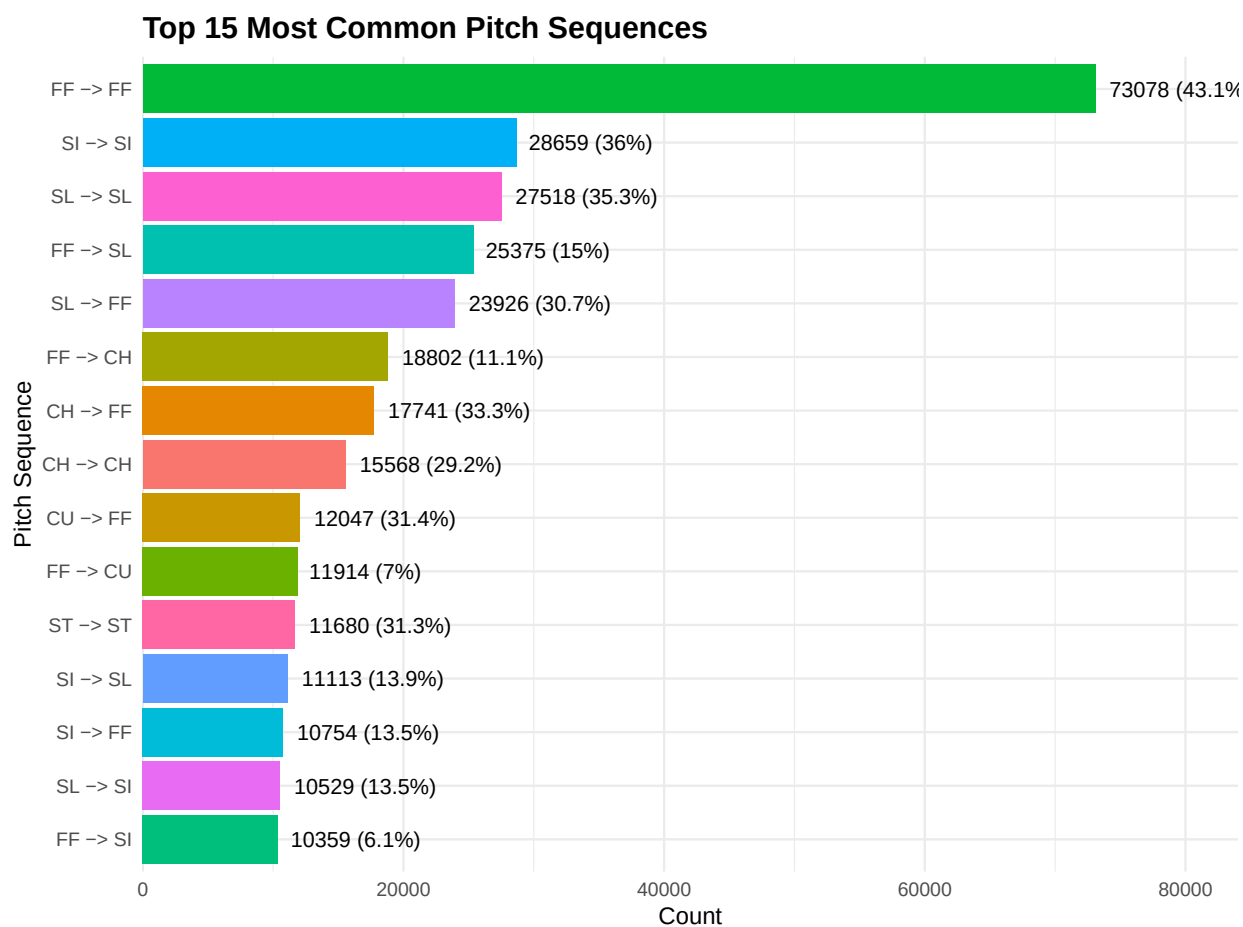


Figure 2.6: Top 15 Most Common Two-Pitch Sequences

Figure 2.6 presents the 15 most frequently observed two-pitch sequences, sorted by raw count. It is important to note that the percentages shown reflect conditional probabilities rather than overall proportions as each percentage represents how often a given pitch type follows the first pitch in that sequence, not the sequence's share of all pitches thrown. For example, FF→FF occurring 73,078 times with a 43.1% label means that after a four-seam fastball is thrown, pitchers follow it with another four-seam fastball 43.1% of the time. Other frequent patterns include sinker-sinker sequences (28,659 times, 36%), meaning sinkers are followed by another sinker 36% of the

time, slider-slider sequences (27,518 times, 35.3%), and fastball-slider sequences (25,375 times, 15%). Because the percentages are conditioned on different first pitches, they cannot be directly compared across sequences, which is why the graph is ordered by count rather than percentage.

2.2.6 Pitch Type by Zone

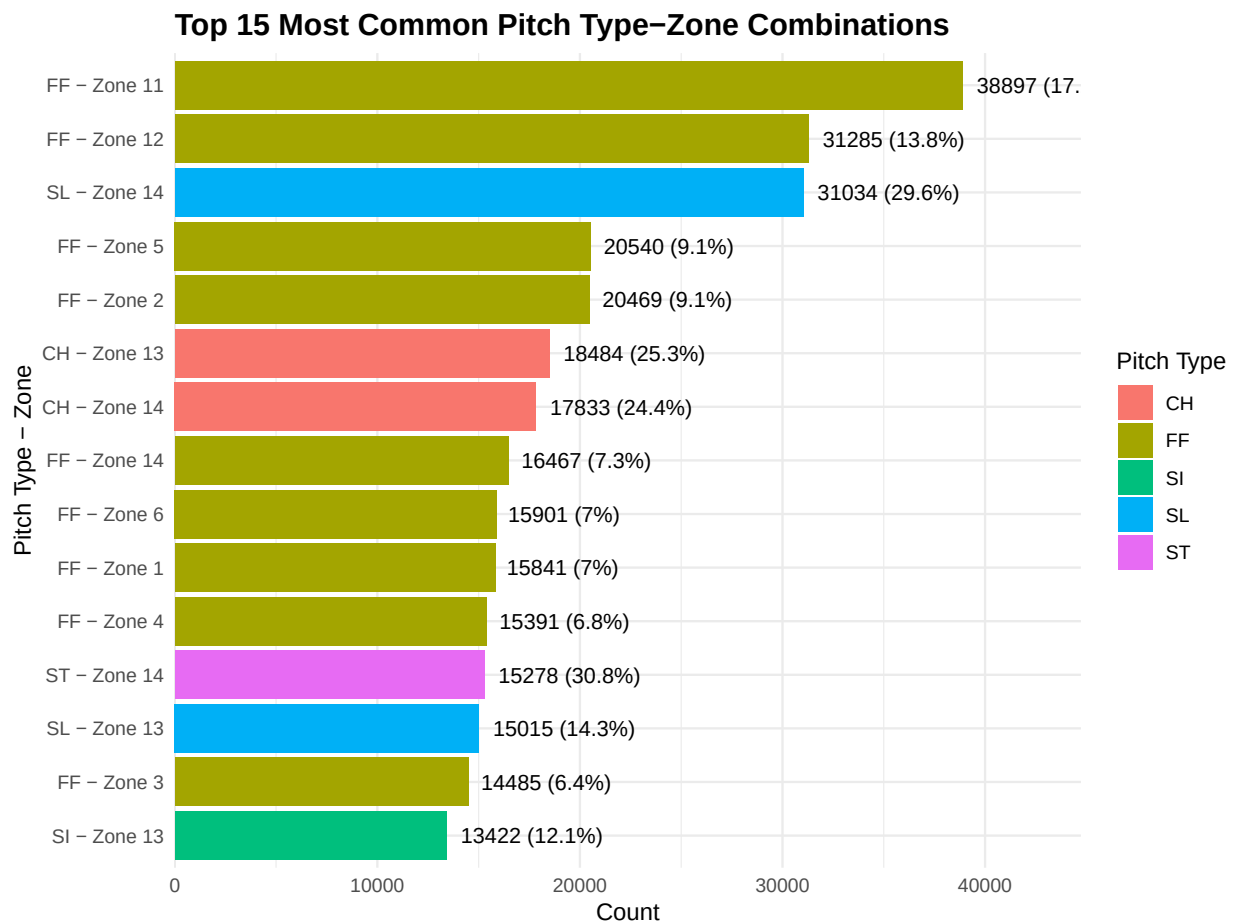


Figure 2.7: Top 15 Most Common Pitch Type-Zone Combinations

Figure 2.7 displays the most frequently observed combinations of pitch type and pitch location zone, sorted by raw count. As with Figure 2.6, the percentages shown are conditional which means each percentage represents how often a given pitch type is thrown to that particular zone

relative to all locations that pitch type is thrown, not the combination's share of all pitches. For example, fastballs thrown to zone 11 represent the single most common pitch-location combination (38,897 times), meaning that of all four-seam fastballs thrown, that percentage land in zone 11 which is consistent with the newer yet conventional strategy of "elevating the fastball." Sliders appear most frequently in zone 14 (outside off the plate, 31,034 occurrences), reflecting their strategic use as "chase" pitches, with that percentage of all sliders being thrown to that location. Changeups concentrate in zones 13 and 14 (low and outside), corresponding to their designed downward movement to induce groundballs. Because percentages are conditioned on each individual pitch type, they cannot be directly compared across different pitch types, which is why the graph is ordered by count rather than percentage.

3. Methodology

This chapter describes the analytical approach employed to predict pitch sequences in Major League Baseball. The research design emphasizes practical implementation and model interpretability while maintaining competitive predictive accuracy.

3.1 Research Design

This research uses a quantitative, predictive modeling approach analyzing pitch sequencing in the 2025 MLB season. Rather than building a single league-wide model, the analytical strategy constructs individual multinomial logistic regression models for each pitcher who threw a sufficient number of pitches during the season.

The research design follows a systematic model comparison framework, evaluating five distinct model specifications of increasing complexity. Each successive model adds additional predictor variables, allowing assessment of whether increased complexity yields proportionate improvements in predictive accuracy. Model performance is evaluated using classification accuracy as the primary metric as the proportion of pitches correctly predicted.

3.2 Theoretical Framework

3.2.1 Multinomial Logistic Regression Framework

Multinomial logistic regression extends binary logistic regression to accommodate outcome variables with more than two categories. In this application, the outcome variable is pitch type, which can take on multiple values (fastball, slider, curveball, changeup, etc.). The model estimates the probability of each possible pitch type conditional on the observed predictor variables.

For an observation i with predictor vector $\mathbf{X}_i$, the probability that the outcome falls into category j is modeled as:

$$P(Y_i = j | \mathbf{X}_i) = \frac{\exp(\mathbf{X}_i \beta_j)}{\sum_{k=1}^J \exp(\mathbf{X}_i \beta_k)}$$

where J represents the total number of pitch type categories, and β_j represents the coefficient vector associated with category j . This formulation ensures that predicted probabilities sum to 1 across all categories.

The model estimates $J-1$ sets of coefficients, with one category serving as the reference group. Coefficients can be interpreted as log-odds ratios. Bascially, a one-unit increase in predictor X changes the log-odds of observing pitch type j relative to the reference category by β_j .

3.2.2 Methodological Innovation: Lagged Variables

A key contribution of this research is demonstrating how multinomial logistic regression can be adapted for sequential prediction through careful incorporation of lagged variables. This

combination addresses a methodological gap in some previous pitch prediction research.

Multinomial logistic regression is optimal for unordered categorical outcomes, like what is being predicted here. It produces interpretable probability distributions rather than hard classifications, and offers coefficient interpretability through log-odds ratios. Unlike neural networks, the model structure remains transparent, allowing analysts to understand why specific predictions are made.

Pitch selection is inherently sequential as pitchers use previous pitches to “set up” subsequent pitches. Lagged variables (`lag1_pitch_type`, `lag2_pitch_type`, `lag1_zone`, `lag2_zone`) implement this sequential dependency by including previous pitches as predictors.

Combining categorical lagged variables with multinomial logistic regression introduces several challenges. The first is parameter expansion as each categorical lag requires multiple indicator variables. Another is at-bat boundary handling as sequences segment at at-bat boundaries. And finally, multiple lag types as this requires the model to learn multi-dimensional sequential patterns. This research demonstrates that classical statistical methods can successfully incorporate sequential dependencies while maintaining interpretability and computational efficiency.

3.2.3 Alternative Approaches Considered

During model development, random forest and XGBoost models were constructed and evaluated using the same predictor variables and training data. However, after initial testing, it was revealed that these sophisticated algorithms provided no meaningful improvement in classification accuracy compared to multinomial logistic regression. Random forest models achieved accuracy to that of logistic regression models, but it required substantially greater computational resources.

This result makes intuitive sense as pitch selection is fundamentally a linear decision process influenced by readily identifiable factors. When the underlying data-generating process is approximately linear, simpler models often match or exceed the performance of complex alternatives while offering superior interpretability.

3.3 Model Specifications

Five distinct model specifications were developed and evaluated, each incorporating different combinations of predictor variables. All models use the same underlying multinomial logistic regression framework, differing only in the predictors included.

The models were developed through an iterative process. Model 3 (Basic Model) was created first as a comprehensive “kitchen sink” approach including all readily observable predictors. Following this baseline, Models 4 and 5 explored alternative ways of modeling the relationship between pitch type and location using interactions and interaction variables. Finally, Models 1 and 2 were developed to test whether simpler specifications could achieve comparable accuracy, while favoring low computational capabilities and ease of implementation.

3.3.1 Model 1: Simple Model

The first model specification represents the minimal viable approach to pitch prediction, incorporating only information about recent pitch history and position within the at-bat.

Predictor variables:

- `lag1_pitch_type`: The type of pitch thrown immediately before the current pitch

- `lag2_pitch_type`: The type of pitch thrown two pitches ago
- `pitch_number`: The sequential number of the current pitch within the at-bat

This specification tests whether simple pitch sequencing patterns alone provide meaningful predictive information.

3.3.2 Model 2: Simple Model with Batter Handedness

The second model extends Model 1 but adds batter handedness:

Predictor variables:

- All variables from Model 1
- `stand`: Whether the batter bats left-handed or right-handed

This addition allows the model to capture platoon related adjustments in pitch selection. This is important as certain pitches move more effectively against same-handed versus opposite-handed batters, not to mention a pitcher's strategy changes based on the whether the batter is left-handed or right-handed.

3.3.3 Model 3: Basic Model with Count and Zone

Model 3 adds onto model 2 but incorporates count information and pitch location history:

Predictor variables:

- All variables from Model 2
- `balls`: Number of balls in the current count

- **strikes:** Number of strikes in the current count
- **lag1_zone:** The pitch location zone of the previous pitch
- **lag2_zone:** The pitch location zone two pitches ago

Count variables capture the strategic dynamic where pitchers facing favorable counts possess greater freedom to throw pitches outside the strike zone, while unfavorable counts necessitate throwing strikes.

3.3.4 Model 4: Separate Type and Zone Models

The fourth approach constructs two separate models rather than a single integrated model to try to predict the next pitch type and the next pitch location:

- **Type Model:** Predicts only pitch type using all predictor variables
- **Zone Model:** Predicts only pitch location using all predictor variables

Both models employ the same predictor set but optimize predictions for their respective outcomes independently. Overall accuracy is computed as the proportion of pitches where both type and zone are predicted correctly.

3.3.5 Model 5: Interaction Model

This model treats pitch type and location as a single joint outcome, predicting specific type-zone combinations rather than modeling them separately.

Predictor variables:

- lag1_pitch_type: Previous pitch type
- lag1_zone: Previous pitch zone
- balls: Current ball count
- strikes: Current strike count
- stand: Batter handedness

Rather than predicting pitch type alone (e.g., “slider”), this model predicts the specific combination (e.g., “slider in zone 14”). This creates a much larger number of outcome categories (potentially 100+ combinations) but captures the interdependence between type and location decisions.

3.4 Model Training and Evaluation

3.4.1 Data Preparation

As explained earlier, prior to model training, lagged variables were constructed by shifting pitch sequence data within each at-bat. It was made sure that lag variables did not carry over between at-bats meaning the first pitch of each at-bat has no previous pitch, so lag variables are coded as “None.”

Categorical variables were properly encoded for inclusion in regression models, with one category designated as the reference level. Observations with missing data for key variables were removed, though the missing data was minimal with less than 0.5% of observations.

3.4.2 Model Fitting Process

For each pitcher who threw more than a minimum threshold of pitches of 100, individual models were fit using that pitcher’s data exclusively. Model fitting uses maximum likelihood estimation implemented through the `nnet` package in R.

The fitting process proceeds iteratively. Starting with initial coefficient estimates, the algorithm computes predicted probabilities, calculates the gradient of the log-likelihood function, and updates coefficients in the direction that increases likelihood. This process repeats until convergence, defined as minimal change in coefficients between iterations.

3.4.3 Performance Evaluation

Classification accuracy is computed as:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

For each pitch, the model generates predicted probabilities for each possible pitch type. The predicted pitch type is the category with the highest probability. A naive baseline model that always predicts the most common pitch type would achieve accuracy equal to that pitch type’s frequency that as discussed earlier, is typically around 30-35%.

3.5 Statistical Comparison of Models

To compare the performance of each model to one another, a few methods were implemented including pairwise comparisons, analysis of variance, and effect size calculations.

3.5.1 Pairwise Comparisons

Model performance was compared using paired t-tests and Wilcoxon signed-rank tests, which account for the fact that multiple models make predictions for the same pitchers. Both tests provided p-values indicating the strength of evidence against the null hypothesis of no difference.

3.5.2 Analysis of Variance

Overall differences across all five models were assessed using one-way repeated measures ANOVA. Following a significant ANOVA result, Tukey’s Honestly Significant Difference post-hoc test identified which specific model pairs differ significantly.

3.5.3 Effect Size Calculation

Cohen’s d was calculated for each significant pairwise comparison:

$$d = \frac{\bar{x}_1 - \bar{x}_2}{s_{\text{pooled}}}$$

Cohen’s d values of 0.2, 0.5, and 0.8 are conventionally interpreted as small, medium, and large effect sizes, respectively.

3.6 Computational Implementation

All data processing, model fitting, and analysis were conducted using R (version 4.3.1 or later). Key packages employed include tidyverse, nnet, caret, and knitr. Total computational time for fitting all five model specifications across all pitchers was approximately 2-4 hours on a com-

puter with standard capabilities.

4. Results

This chapter presents the performance evaluation and comparative analysis of the five multinomial logistic regression model specifications. Results are organized into four major sections: overall model performance comparison, statistical significance testing, pitcher-specific performance patterns, and model complexity analysis. Findings reveal substantial differences in predictive accuracy across model specifications and considerable variation in predictability across individual pitchers.

4.1 Overall Model Performance

4.1.1 Summary Statistics

Table 4.1: Overall Model Performance Summary

Model	N Pitchers	Mean	Median	SD
		Accuracy	Accuracy	
Model 3: Basic	767	0.6236	0.5782	0.1797
Model 2: Simple + Stand	767	0.4975	0.4941	0.1070
Model 1: Simple	767	0.4720	0.4637	0.1125
Model 4: Separate - Combined	767	0.3432	0.1894	0.3044

(Approximated)

Model	N Pitchers	Mean	Median	SD
		Accuracy	Accuracy	
Model 5: Interaction	596	0.2310	0.2130	0.0811

Table 4.1 displays the distribution of classification accuracy across all pitchers for each of the five model configurations. Model 3 (Basic) achieved the highest mean accuracy at 62.36% (median = 57.82%, SD = 0.1797), making it the best performing model for predicting pitch type. This result is particularly notable given that Model 4's Type Only component, which uses an identical predictor structure to Model 3 but is optimized solely for pitch type prediction, achieved a mean accuracy of 62.38%, which was a difference of only 0.02 percentage points. Because these two models are functionally equivalent for pitch type prediction, Model 4's Type Only output was excluded from this table to avoid redundancy. Instead, Model 4 is represented here by its approximated combined accuracy, which reflects its intended purpose of jointly predicting both pitch type and location.

The simple models (Models 1 and 2) achieved substantially lower accuracy than Model 3. Model 1 (Simple) reached 47.20% mean accuracy (median = 46.37%, SD = 0.1125), while Model 2 (Simple + Stand) improved to 49.75% (median = 49.41%, SD = 0.1070). The addition of batter handedness to the simple model yielded a modest but consistent improvement of 2.55 percentage points, suggesting that platoon considerations contribute meaningfully to pitch selection even in the absence of count information. The similar standard deviations between Models 1 and 2 indicate that adding batter handedness improves accuracy consistently across pitchers rather than benefiting only a subset.

Model 4 (Separate - Combined Approximated) achieved a mean accuracy of 34.32% (me-

dian = 18.94%, SD = 0.3044), representing the estimated probability of correctly predicting both pitch type and location zone simultaneously. This figure is an approximation calculated by multiplying each pitcher's Type Only and Zone Only accuracies under the assumption that the two predictions are independent meaning the true combined accuracy would require raw per-pitch prediction data. The extremely large standard deviation (0.3044) and the stark gap between the mean (34.32%) and median (18.94%) indicate severe right-skew in the distribution. This indicates most pitchers have quite low combined accuracy, while a small number of pitchers with highly predictable tendencies in both type and location achieve much higher combined scores. This wide variability reflects the compounding nature of joint prediction as a pitcher must be predictable in both dimensions simultaneously to achieve a high combined accuracy.

Model 5 (Interaction) performed substantially worse than all other specifications, achieving only 23.10% mean accuracy (median = 21.30%, SD = 0.0811). This poor performance reflects the fundamental challenge of predicting joint type-zone outcomes within a single model, where the vast increase of outcome categories (over 100 possible pitch type and zone combinations) creates severe data sparsity. With many type-zone combinations occurring only rarely in a pitcher's season, the model struggles to learn reliable patterns for infrequent events. The narrow standard deviation relative to its mean indicates that this failure is consistent across pitchers, suggesting a structural limitation of the approach rather than a pitcher-specific issue.

4.1.2 Visualization of Model Performance

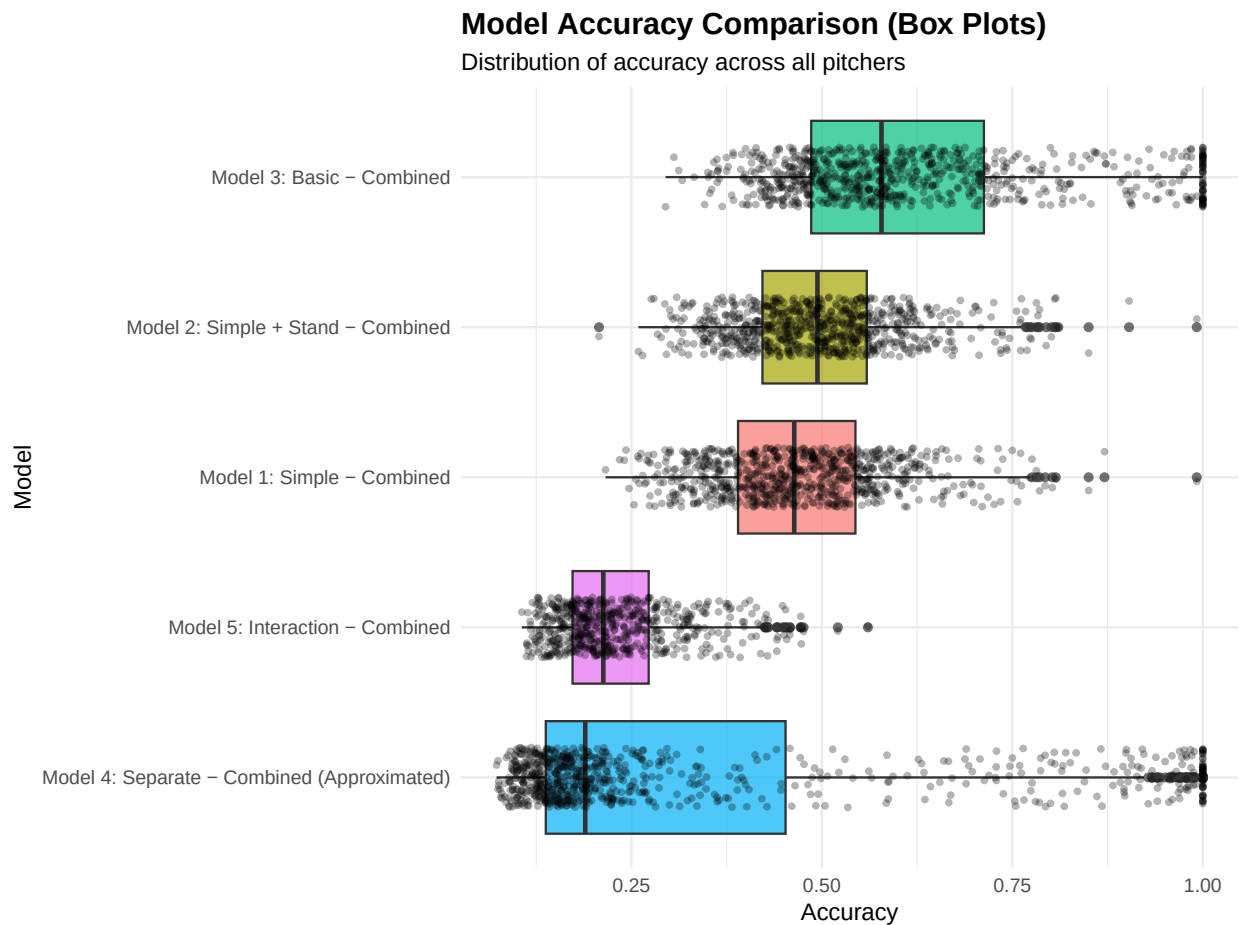


Figure 4.1: Model Accuracy Comparison Across All Pitchers

Figure 4.1 presents boxplots showing substantial variation both across models and within models across different pitchers. The simple models (Models 1 and 2) show relatively narrow, concentrated distributions centered around 45–50% accuracy. This tight clustering indicates that these models perform consistently across pitchers. While absolute accuracy is moderate, the models are reliable in producing similar accuracy regardless of the pitcher being modeled. The limited spread likely reflects these models' reliance on pitch sequencing patterns and batter handedness, which exhibit moderate consistency across pitchers. The interquartile ranges for both models are similarly

compact, and the jittered points confirm that few pitchers fall far outside the central tendency.

Model 3 (Basic) displays a much broader distribution, with substantial spread spanning from approximately 30% to over 90% accuracy. This wide variation indicates that the comprehensive model performs very well for some pitchers but struggles with others. The large interquartile range and long right tail suggest that count and location information is highly informative for a subset of pitchers. That subset of pitchers are likely those with strong, consistent tendencies tied to game state, while providing less benefit for pitchers whose selection patterns are less structured. The median sits near 57.82%, but the broad spread of jittered points above 75% confirms that a meaningful number of pitchers are highly predictable under this model.

Model 4 (Separate - Combined Approximated) exhibits the widest distribution of any model, with a very low median near 18.94% but a long right tail extending all the way to 1.0. The interquartile box sits at the low end of the accuracy range, yet the scatter of jittered points across the full range illustrates the high variability inherent in multiplying two independent accuracy values. Pitchers who are simultaneously predictable in both pitch type and location achieve very high combined scores, while those with only moderate accuracy in either dimension see their combined score drop substantially due to the multiplicative penalty.

Model 5 (Interaction) shows a narrow distribution concentrated at low accuracy values around 20–25%, with virtually no observations above 40%, confirming that the fundamental structural limitation of joint type-zone prediction affects all pitchers similarly regardless of individual tendencies. The tight clustering here mirrors Models 1 and 2 in spread, but at a far lower accuracy level.

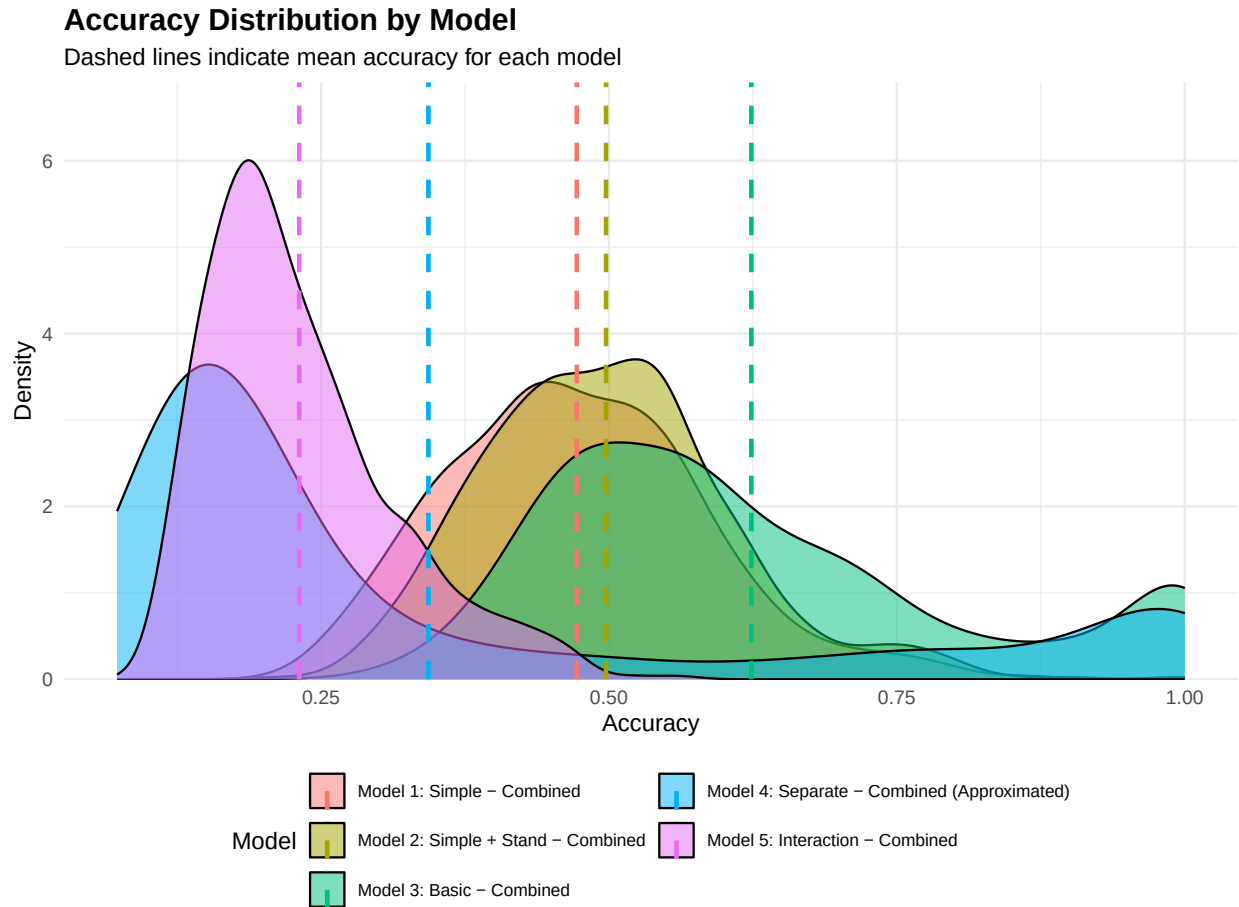


Figure 4.2: Accuracy Distribution by Model

Figure 4.2’s density plots further illustrate the distributional differences across models. Model 5 (Interaction) shows a sharp, narrow peak around 0.20–0.25, confirming consistently poor performance across pitchers with little variation. Model 4 (Separate - Combined Approximated) displays a similarly left-skewed distribution with high density at low accuracy values and a long flat tail extending toward 1.0. Models 1 and 2 show moderately tight distributions centered around 0.47 and 0.50 respectively, with their mean lines closely aligned with their density peaks. Model 3 (Basic) exhibits the broadest and most spread out distribution, with its mean at 0.6236, reflecting the wide variation in how well the full model captures individual pitcher tendencies. The separation between the dashed mean lines across models visually confirms the performance hierarchy

established in Table 4.1.

4.2 Statistical Significance Testing

```
      Df Sum Sq Mean Sq F value Pr(>F)
model      4  62.37   15.591    483.2 <2e-16 ***
Residuals 3659 118.07    0.032
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Tukey multiple comparisons of means
95% family-wise confidence level

Fit: aov(formula = accuracy ~ model, data = accuracy_comparison)

$model

              diff              lwr
Model 2: Simple + Stand-Model 1: Simple    0.02541863  0.0003840616
Model 3: Basic-Model 1: Simple             0.15160320  0.1265686346
Model 4: Separate-Model 1: Simple          -0.12881763 -0.1538521987
Model 5: Interaction-Model 1: Simple        -0.24100994 -0.2677800282
Model 3: Basic-Model 2: Simple + Stand      0.12618457  0.1011500092
Model 4: Separate-Model 2: Simple + Stand  -0.15423626 -0.1792708241
Model 5: Interaction-Model 2: Simple + Stand -0.26642856 -0.2931986536
```

Model 4: Separate-Model 3: Basic	-0.28042083	-0.3054553971
Model 5: Interaction-Model 3: Basic	-0.39261314	-0.4193832267
Model 5: Interaction-Model 4: Separate	-0.11219230	-0.1389623934

	upr	p adj
Model 2: Simple + Stand-Model 1: Simple	0.05045319	0.044531
Model 3: Basic-Model 1: Simple	0.17663776	0.000000
Model 4: Separate-Model 1: Simple	-0.10378307	0.000000
Model 5: Interaction-Model 1: Simple	-0.21423985	0.000000
Model 3: Basic-Model 2: Simple + Stand	0.15121914	0.000000
Model 4: Separate-Model 2: Simple + Stand	-0.12920170	0.000000
Model 5: Interaction-Model 2: Simple + Stand	-0.23965847	0.000000
Model 4: Separate-Model 3: Basic	-0.25538627	0.000000
Model 5: Interaction-Model 3: Basic	-0.36584305	0.000000
Model 5: Interaction-Model 4: Separate	-0.08542221	0.000000

Model 1: Simple vs Model 2: Simple + Stand: Cohen's d = -0.232

Effect size: Medium

Model 1: Simple vs Model 3: Basic: Cohen's d = -1.011

Effect size: Large

Model 1: Simple vs Model 5: Interaction: Cohen's d = 2.333

Effect size: Large

Model 1: Simple vs Model 4: Separate: Cohen's d = 0.561

Effect size: Large

Model 2: Simple + Stand vs Model 3: Basic: Cohen's $d = -0.853$

Effect size: Large

Model 2: Simple + Stand vs Model 5: Interaction: Cohen's $d = 2.729$

Effect size: Large

Model 2: Simple + Stand vs Model 4: Separate: Cohen's $d = 0.676$

Effect size: Large

Model 3: Basic vs Model 5: Interaction: Cohen's $d = 3.413$

Effect size: Large

Model 3: Basic vs Model 4: Separate: Cohen's $d = 1.122$

Effect size: Large

Model 5: Interaction vs Model 4: Separate: Cohen's $d = 0.428$

Effect size: Medium

4.2.1 Analysis of Variance Results

One-way ANOVA testing for overall differences across the five model configurations revealed extremely strong evidence of performance differences ($F(4, 3659) = 483.2, p < 2 \times 10^{-16}$). This highly significant result confirms that model specification substantially affects prediction accuracy, with differences far too large to be attributable to chance variation. The large F-statistic and near-zero p-value confirm that at least some models differ meaningfully in performance, validating the model comparison approach and justifying proceeding to pairwise comparisons.

4.2.2 Tukey's HSD Post-Hoc Comparisons

Tukey's Honestly Significant Difference test, which controls family-wise error rate across multiple comparisons, identified significant differences for all ten possible model pairs. Every pairwise comparison yielded p-values well below 0.05, indicating that each model's performance differs significantly from every other model's performance. Here are seven of the notable model comparisons.

Model 1 vs. Model 2 (Simple vs. Simple + Stand): Model 2 outperformed Model 1 by 2.54 percentage points on average (95% CI: [0.004, 0.050], $p = 0.045$). This difference, while statistically significant, represents a modest improvement. The narrow confidence interval indicates reasonable precision in this estimate.

Model 1 vs. Model 3 (Simple vs. Basic): Model 3 outperformed Model 1 by 15.16 percentage points on average (95% CI: [0.127, 0.177], $p < 0.001$). This substantial difference demonstrates the considerable value of incorporating count and location information, representing a roughly 32% relative improvement over Model 1's baseline accuracy.

Model 2 vs. Model 3 (Simple + Stand vs. Basic): Model 3 outperformed Model 2 by 12.62 percentage points (95% CI: [0.101, 0.151], $p < 0.001$). This isolates the contribution of count and location information beyond what simple sequencing and handedness provide, demonstrating that game state information adds substantial predictive value.

Model 1 vs. Model 4 (Simple vs. Separate - Combined Approximated): Model 1 outperformed Model 4 by 12.88 percentage points (95% CI: [0.104, 0.154], $p < 0.001$), reflecting the difficulty of jointly predicting both pitch type and location simultaneously.

Model 3 vs. Model 4 (Basic vs. Separate - Combined Approximated): Model 3 outper-

formed Model 4 by 28.04 percentage points (95% CI: [0.255, 0.305], $p < 0.001$), representing the largest performance gap in any comparison involving Model 4 and underscoring that requiring simultaneous correctness on both type and zone dramatically reduces overall accuracy.

Model 3 vs. Model 5 (Basic vs. Interaction): Model 3 outperformed Model 5 by 39.26 percentage points (95% CI: [0.366, 0.419], $p < 0.001$), the largest performance gap observed in any comparison and highlighting the severe limitations of the joint type-zone interaction approach.

Model 4 vs. Model 5 (Separate - Combined Approximated vs. Interaction): Model 4 outperformed Model 5 by 11.22 percentage points (95% CI: [0.085, 0.139], $p < 0.001$), indicating that even the approximated combined accuracy of the separate model meaningfully exceeds the interaction model's performance.

4.2.3 Effect Size Analysis

Cohen's d values confirm that differences are both statistically detectable and practically meaningful across most comparisons. Model 1 vs. Model 2 yielded $d = -0.232$, a small-to-medium effect indicating that while batter handedness contributes statistically, its practical impact is modest. Model 1 vs. Model 3 ($d = -1.011$) and Model 2 vs. Model 3 ($d = -0.853$) both produced large effect sizes, confirming that the addition of count and location information yields substantial real-world improvements. Model 3 vs. Model 4 ($d = 1.122$) and Model 3 vs. Model 5 ($d = 3.413$) produced the largest effect sizes, underscoring that Model 3 dramatically outperforms both joint-prediction approaches. Model 4 vs. Model 5 yielded $d = 0.428$, a medium effect, suggesting that while both approaches struggle with joint prediction, the separate model's approximated combined accuracy meaningfully exceeds the interaction model. Models 1 and 2 versus Model 4 both produced large

effect sizes ($d = 0.561$ and $d = 0.676$ respectively), reflecting that even simple models outperform the joint prediction approach on a combined accuracy basis.

4.3 Pitcher-Specific Performance

Table 4.2: Top 10 Most Predictable Pitchers (Highest Mean Accuracy)

	N	Mean	Min	Max			
Pitcher ID	Models	Accuracy	SD	Acc	Acc	Range	Total Pitches
Kahnle, Tommy	4	0.7026	0.2970	0.2571	0.8530	0.5960	1037
Hill, Tim	4	0.6756	0.2623	0.2822	0.8163	0.5342	912
Jansen, Kenley	4	0.6425	0.3099	0.1778	0.8048	0.6270	878
Mattson, Isaac	4	0.6363	0.2752	0.2236	0.7755	0.5520	786
Pomeranz, Drew	4	0.6296	0.2810	0.2086	0.7883	0.5797	805
Rogers, Tyler	4	0.6156	0.2671	0.2154	0.7666	0.5511	978
Palencia, Daniel	4	0.5920	0.2459	0.2235	0.7296	0.5061	845
Johnson, Pierce	4	0.5915	0.2338	0.2408	0.7085	0.4677	979
Sabrowski, Erik	4	0.5795	0.2288	0.2381	0.7254	0.4873	503
Clase, Emmanuel	4	0.5791	0.2197	0.2504	0.7100	0.4597	754

Table 4.2 presents the ten most predictable pitchers in the dataset with a minimum of 500

pitches thrown, ranked by mean accuracy across all models. Tommy Kahnle leads with 70.26% mean accuracy, followed by Tim Hill (67.56%) and Kenley Jansen (64.25%). Notably, this list is dominated by relief pitchers as Kahnle, Jansen, Hill, Rogers, Clase, and Johnson are all bullpen arms, while Pomeranz, Mattson, Palencia, and Sabrowski are also primarily relievers. This pattern is likely not coincidental. Relief pitchers typically have narrower pitch repertoires than starters, often relying heavily on one or two primary pitches, which makes their selection patterns more systematic and therefore more predictable to a statistical model. Additionally, relief pitchers frequently enter games in high-leverage, specific situations as closers like Clase and Jansen are often deployed in defined roles which may further limit their pitch selection in predictable ways.

The standard deviations for most pitchers in this group are notably high, ranging from 0.2197 (Clase) to 0.3099 (Jansen), indicating that accuracy varies considerably depending on which model is used. This suggests that while these pitchers are genuinely predictable, the degree to which any given model captures their patterns depends heavily on model specification. Even so, the minimum accuracies for pitchers like Kahnle (25.71%) and Jansen (17.78%) show that not all models succeed equally, and the approximated Model 4 combined accuracy likely drives these lower bounds given its multiplicative penalty.

Table 4.3: Top 10 Least Predictable Pitchers (Lowest Mean Accuracy)

	N	Mean		Min	Max		
Pitcher ID	Models	Accuracy	SD	Acc	Acc	Range	Total Pitches
Cameron, Noah	4	0.2630	0.1029	0.1201	0.3644	0.2444	2167
Vásquez, Randy	4	0.2616	0.1047	0.1180	0.3635	0.2456	2153
Quantrill, Cal	4	0.2611	0.1005	0.1354	0.3770	0.2416	2006

	N	Mean		Min	Max		
Pitcher ID	Models	Accuracy	SD	Acc	Acc	Range	Total Pitches
Cabrera, Edward	4	0.2607	0.0965	0.1224	0.3461	0.2237	2253
Alcantara,	4	0.2549	0.0999	0.1148	0.3510	0.2363	2893
Sandy							
Martinez, Nick	4	0.2538	0.0980	0.1231	0.3504	0.2273	2582
Buehler, Walker	4	0.2426	0.0728	0.1402	0.3120	0.1718	2191
Lorenzen,	4	0.2353	0.0853	0.1126	0.3057	0.1931	2318
Michael							
Darvish, Yu	4	0.2288	0.0610	0.1754	0.3165	0.1411	1160
Lugo, Seth	4	0.2265	0.0767	0.1176	0.2947	0.1772	2381

Table 4.3 presents the ten least predictable pitchers, all of whom are starting pitchers with substantial pitch counts ranging from 1,160 (Darvish) to 2,893 (Alcantara) pitches in the dataset. This stands in direct contrast to the most predictable group and reinforces the relief pitcher vs. starter dynamic observed above. Seth Lugo (22.65%), Yu Darvish (22.88%), and Michael Lorenzen (23.53%) show the lowest mean accuracy values, barely exceeding the random guessing baseline for a multi-class outcome. The presence of accomplished, experienced starters in this group like Darvish, Sandy Alcantara, Walker Buehler, and Cal Quantrill among them, suggests that elite starting pitchers are genuinely difficult to model, likely because they maintain diverse repertoires, adjust sequencing based on batter tendencies, and deliberately avoid falling into predictable patterns across a long outing.

The accuracy ranges for unpredictable pitchers are strikingly narrow compared to the most

predictable group. Darvish’s accuracy spans only 14.11 percentage points (17.54% to 31.65%), and Lugo’s spans just 17.72 percentage points (11.76% to 29.47%), indicating that no model performs well for these pitchers regardless of specification. This consistency in poor performance across all four models suggests that these pitchers’ unpredictability is a genuine characteristic of their approach rather than a limitation any particular model can overcome. Taken together, these two tables suggest that the most meaningful predictor of pitcher predictability in this dataset may not be age or experience, but rather role as relief pitchers with defined, narrow repertoires are systematically more predictable than starters who must vary their approach across many batters and innings.

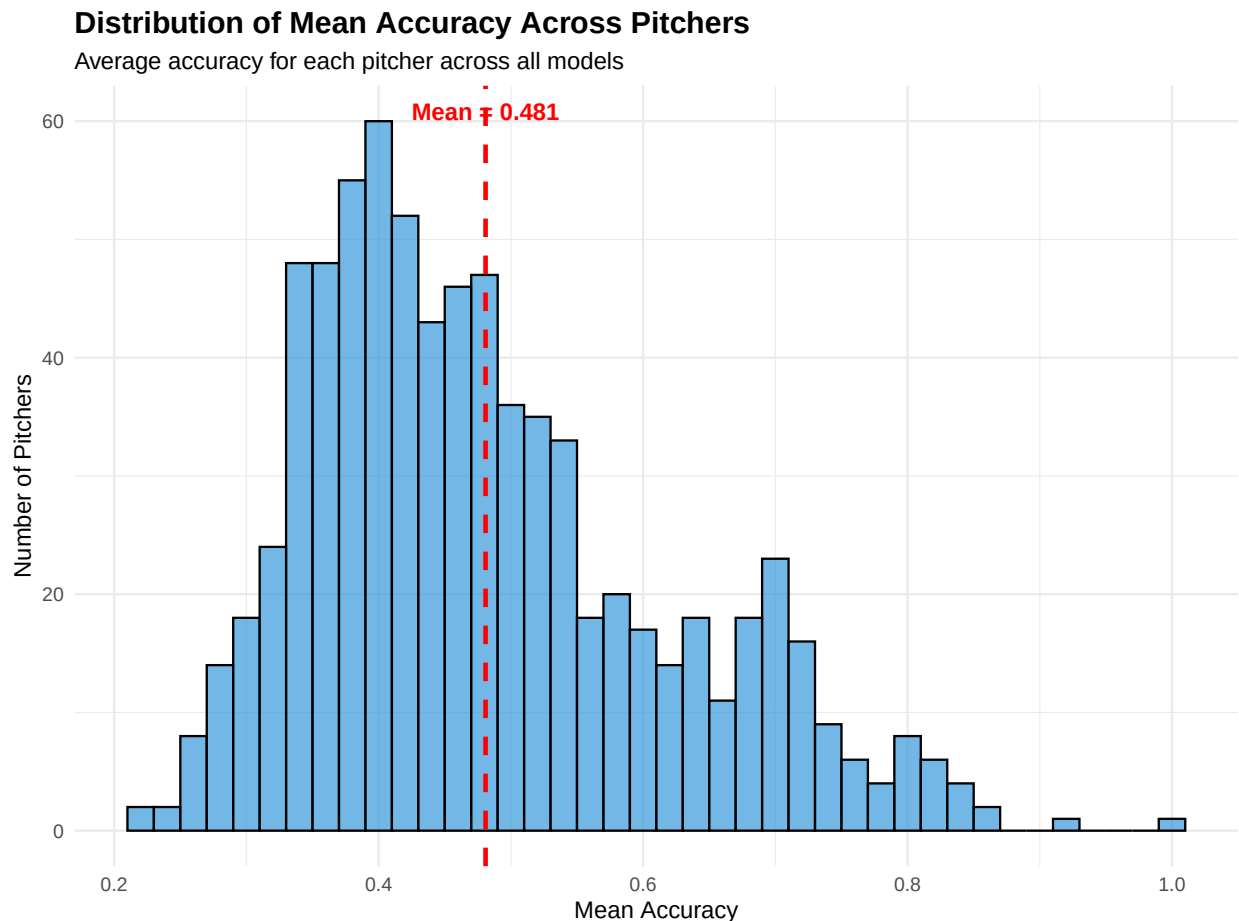


Figure 4.3: Distribution of Mean Accuracy Across Pitchers

Figure 4.3 presents a histogram of mean accuracy across pitchers, where each pitcher’s

mean represents their average accuracy across all models for which they have predictions. The overall mean accuracy across all pitchers is 48.09% (median = 45.60%, SD = 13.53%), indicating substantial diversity in pitcher predictability.

The distribution appears approximately normal with slight right-skew, centered around 45-50% accuracy. Approximately 60% of pitchers fall within one standard deviation of the mean (34.5% to 61.6% accuracy). This relatively tight clustering around the middle of the distribution suggests that most pitchers exhibit moderate predictability as neither extremely systematic nor completely random in their pitch selection patterns. The right tail extends to several pitchers achieving mean accuracy above 80%, while the left tail includes pitchers below 30%, reflecting genuine differences in how pitchers approach sequencing.

Correlation between mean accuracy and SD: 0.5657 (p-value: 0)

The correlation between mean accuracy and standard deviation across pitchers is 0.566 ($p < 0.001$), indicating a moderate positive relationship: more predictable pitchers tend to show greater variation across models. This pattern makes intuitive sense as when clear patterns exist, different models may capture different aspects of those patterns, leading to varying performance. When no clear patterns exist, all models perform poorly and similarly, yielding low standard deviation. This finding confirms that model choice matters primarily when systematic behavior exists to be modeled.

4.4 Model Complexity vs. Performance

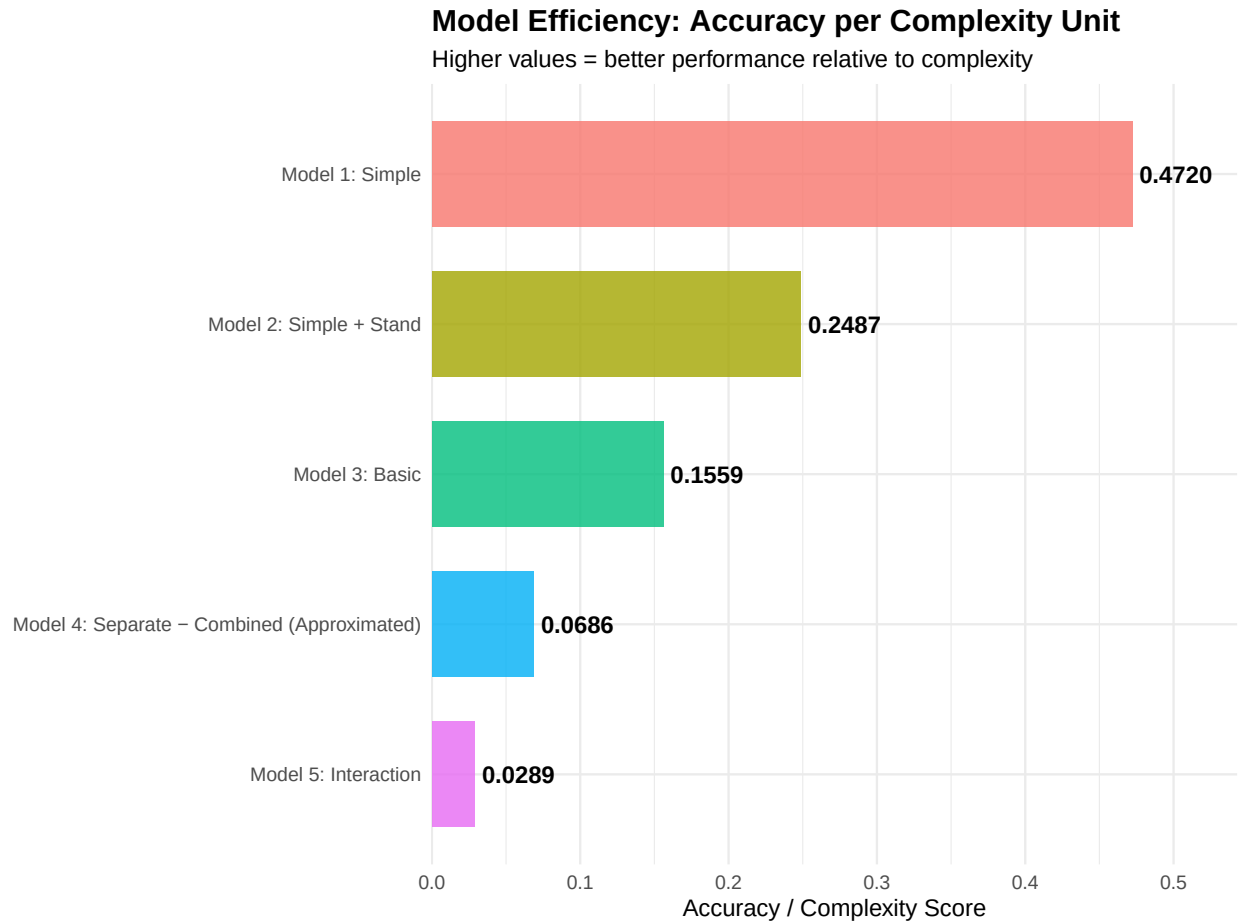


Figure 4.4: Model Efficiency - Accuracy per Complexity Unit

Figure 4.4 presents model efficiency, calculated as mean accuracy divided by complexity score. Complexity scores were assigned based on the number of distinct predictor features and overall model structure: Model 1 (3 features) = complexity 1, Model 2 (4 features) = complexity 2, Model 3 (7 features) = complexity 4, Model 4 (7 features, two separate models) = complexity 5, and Model 5 (10 features, joint outcome) = complexity 8. Model 1 achieves the highest efficiency score (0.4720), indicating the best performance relative to its complexity. Despite its lower abso-

lute accuracy (47.20%), its minimal complexity yields a strong efficiency ratio. Model 2 shows moderate efficiency (0.2487), while Model 3 achieves an efficiency of 0.1559 despite its highest absolute pitch-type accuracy. Model 4's approximated combined efficiency (0.0686) reflects the compounding penalty of requiring simultaneous correctness of two predictions. Model 5 achieves the lowest efficiency score (0.0289), confirming that its complexity investment yields almost no return in predictive accuracy.

This analysis highlights a fundamental tradeoff: Model 3 achieves the best absolute accuracy for pitch type prediction (62.36%) but requires substantial complexity, while Model 1 achieves moderate accuracy (47.20%) with minimal complexity. Model 4's purpose with jointly predicting both type and zone, introduces an intrinsic accuracy cost that its efficiency score reflects. For practical applications where real-time implementation is critical and interpretability is valued, Model 1's high efficiency may favor it despite lower accuracy. For applications where maximum pitch type accuracy is key and computational resources are available, Model 3's superior absolute performance justifies its complexity.

Recommendations:

- **For Maximum Accuracy:** Model 3 (62.36% accuracy), with large effect sizes justifying the added complexity
- **For Best Efficiency:** Model 1 (47.20% accuracy), optimal for real-time implementation
- **For Joint Type and Zone Prediction:** Model 4 Separate (34.32% approximated combined accuracy), the only model designed for this purpose
- **Avoid:** Model 5 (23.10% accuracy), which exhibits catastrophically poor performance at the highest complexity cost

5. Discussion

This chapter interprets the findings presented in the Results section, discusses their implications for baseball analytics and practice, acknowledges limitations, and identifies directions for future research.

5.1 Key Findings Summary

This research developed and evaluated five multinomial logistic regression model specifications for predicting pitch sequences in Major League Baseball. Analysis of 712,528 pitches from the 2025 MLB season across 767 pitchers yielded several key findings.

First, Model 3 (Basic) achieved the highest prediction accuracy for pitch type at 62.36%, substantially outperforming simpler alternatives. This comprehensive model, which incorporates count information, previous pitch types, previous pitch locations, and batter handedness, demonstrates that pitch selection is primarily driven by game state, particularly the count, rather than purely by sequential patterns.

Second, simple models achieved adequate but lower performance. Model 1 reached 47.20% accuracy and Model 2 improved to 49.75%, both substantially exceeding random guessing baselines of 30–35%. These results demonstrate that pitch sequencing patterns alone contain meaningful predictive information. The 2.55 percentage point improvement from adding handedness validates platoon considerations, though count information contributes more substantially (12.62

additional points from Model 2 to Model 3).

Third, Model 5 (Interaction) failed catastrophically, achieving only 23.10% accuracy which was worse than random guessing for most pitchers. This failure showed fundamental constraints on model complexity when outcome categories increase. The joint modeling of pitch type and location created 100+ distinct outcome categories, leading to severe data sparsity that prevented reliable parameter estimation.

Fourth, pitcher predictability varies enormously, with mean accuracy ranging from approximately 25% to over 70% among pitchers with at least 500 pitches thrown. This spread is strongly associated with pitcher role rather than age or experience. Relief pitchers with narrow repertoires, such as Tommy Kahnle and Kenley Jansen, dominate the most predictable group, while experienced starting pitchers with diverse arsenals, such as Yu Darvish and Seth Lugo, occupy the least predictable group. This pattern suggests that strategic unpredictability among starters may represent a developed skill rather than merely a failure to establish patterns.

Finally, statistical testing confirmed that performance differences are both significant and meaningful. ANOVA revealed overwhelming evidence of differences across models ($F(4, 3659) = 483.2, p < 2 \times 10^{-16}$), while effect size analysis confirmed large practical differences: Model 3 versus Model 1 yielded $d = -1.011$ (large effect), while Model 3 versus Model 5 produced $d = 3.413$ (enormous effect).

5.2 Interpretation

5.2.1 Why Model 3 Dominates

Model 3’s superior performance validates the theoretical framework emphasizing count-dependent pitch selection as baseball’s primary strategic dimension. When pitchers face unfavorable counts (more balls than strikes), strategic imperatives force them toward reliable strike-throwing pitches; typically fastballs. Conversely, favorable counts grant freedom to deploy full arsenals, including pitches that might miss the strike zone. These count-driven patterns create predictable behavior that Model 3 exploits by learning pitcher-specific count strategies. For example, how aggressively each pitcher leans on fastballs when behind, how extensively they attack with breaking balls when ahead, and how their sequencing patterns interact with count state.

Model 3’s broad accuracy distribution across pitchers reveals that its value depends heavily on pitcher type. For relief pitchers with systematic, count-based strategies and narrow repertoires, it provides substantial advantages. For starting pitchers who deliberately vary their approach across many batters and innings, its additional complexity yields comparatively less benefit, though it still outperforms simpler models on average.

5.2.2 Why Simple Models Perform Adequately

Models 1 and 2’s respectable performance (47-50% accuracy) demonstrates that pitch sequencing patterns exist independently of count considerations. Baseball conventional wisdom about “setting up” hitters by using pitch sequences to disrupt timing and create deception, finds empirical support in these models’ above-baseline accuracy. The simple models’ narrow, con-

centrated accuracy distributions indicate consistent performance across diverse pitcher types, suggesting that basic sequencing patterns like fastballs followed by breaking balls, breaking balls in clusters, systematic first-pitch tendencies, occur with moderate regularity across the league regardless of individual pitcher characteristics.

Model 2's modest improvement over Model 1 (2.55 points) through adding handedness confirms platoon effects while revealing their secondary importance relative to count. Pitchers will adjust arsenals based on batter handedness, but these adjustments represent refinements rather than primary determinants of pitch selection. For applications requiring rapid computation, easy interpretation, or deployment with limited data infrastructure, these models offer viable alternatives despite lower absolute accuracy.

5.2.3 Why the Interaction Model Failed

Model 5's catastrophic failure (23.10% accuracy) reveals fundamental limits on model complexity in the presence of data sparsity. With 7-10 pitch types and 14 zones, potentially 98-140 distinct type-zone combinations exist. Individual pitchers produce 80-100 combinations over a season, but many occur only 5-10 times which is insufficient for stable parameter estimation. Maximum likelihood estimation struggles with rare outcome categories, yielding parameter estimates that are essentially random rather than systematically related to predictors.

The increase of parameters worsens this problem. Each predictor requires a coefficient for each outcome category, creating potentially thousands of parameters to estimate from only 2,000-3,000 pitch observations per pitcher. This parameter-to-observation ratio violates fundamental statistical principles about sample size requirements for stable estimation. The theoretical motivation

for joint type-location modeling was sound as pitch type and location clearly relate, but practical implementation encountered insurmountable obstacles. This failure demonstrates that more information and refined relationship modeling can backfire when complexity increases faster than data availability.

5.2.4 Pitcher-Specific Patterns

The variation in predictability across pitchers reflects genuine strategic diversity rather than simple sampling variation. The most striking finding is the role-based divide between relief pitchers and starters. Relief pitchers dominate the most predictable group, likely because they carry narrower pitch repertoires, enter games in defined high-leverage situations, and often rely on one or two elite pitches rather than sequencing across a full arsenal. Closers like Emmanuel Clase and Kenley Jansen are deployed in specific contexts that constrain their pitch selection in ways statistical models can exploit. Conversely, starting pitchers dominate the least predictable group. Elite veterans like Yu Darvish and Sandy Alcantara who both have substantial pitch counts confirmed this is not a sample size artifact, as they maintain diverse repertoires and adjust sequencing dynamically across a full outing, making their behavior far harder to model. The presence of these accomplished starters among the least predictable pitchers supports the interpretation that unpredictability is a cultivated competitive skill rather than a random characteristic.

The moderate positive correlation between mean accuracy and standard deviation across models ($r = 0.566$) provides additional insight. When clear patterns exist, different modeling approaches capture different pattern aspects with varying success, yielding high standard deviation. When no clear patterns exist, all models fail similarly, producing low standard deviation despite

low mean accuracy. This confirms that model choice matters primarily when organized behavior exists.

5.3 Practical Applications

5.3.1 Selective Implementation by Opponent

Rather than deploying predictive models universally, teams could implement them selectively based on both opponent predictability and pitcher role. Pre-game analysis identifying relief pitchers where Model 3 accuracy exceeds 60-70% would indicate the most worthwhile targets for real-time prediction, while starting pitchers, who tend to be less predictable and adjust dynamically across outings, may warrant a more conservative deployment threshold.

5.3.2 Pre-Game Scouting Enhancement

Models could generate comprehensive scouting reports before games, condensed into simple decision rules that hitters memorize. For example, “in two-strike counts after throwing two consecutive fastballs, Pitcher X throws sliders 65% of the time.” These memorable patterns could allow hitters to prepare mentally without requiring technological assistance during at-bats. Transforming complex statistical models into instinctively applicable rules of thumb could be useful to comply with current MLB in-game technology rules.

5.3.3 Pitcher Self-Analysis and Development

Pitchers and pitching coaches could use these models diagnostically to assess their own predictability. A pitcher discovering they are highly predictable (Model 1 achieves 70%+ accuracy) might intentionally introduce more randomness into their sequencing. Conversely, a developing pitcher struggling with consistency might benefit from becoming more systematic, establishing reliable sequencing patterns that allow better execution planning. This self-analysis inverts the traditional use case. As rather than exploiting opponent predictability, pitchers use the model to evaluate and improve their own strategic approach.

5.4 Limitations

5.4.1 Data and Temporal Limitations

This research analyzed only the 2025 MLB season, providing substantial data (712,528 pitches) but that makes it impossible to tell if the findings hold up over time. Do individual pitchers maintain consistent sequencing patterns across multiple seasons, or do they evolve their approaches over time? Single-season analysis cannot address these questions about pattern persistence and strategic evolution. The pitcher-specific modeling approach also creates sample size challenges for pitchers near minimum thresholds, where parameter estimates may be less stable than for high-workload pitchers with 3,000+ pitches. Additionally, the decision to filter pitcher-specific performance tables to pitchers with a minimum of 500 pitches excludes a portion of the dataset from individual-level analysis. While this threshold ensures more stable model estimates, it means that findings about pitcher predictability reflect only those with sufficient workload and

may not generalize to pitchers at the margins of MLB rosters.

5.4.2 Methodological Limitations

Classification accuracy treats all prediction errors equally: predicting a curveball when a slider was thrown costs the same as predicting a curveball when a fastball was thrown, despite these errors having very different strategic implications. Alternative metrics like logarithmic loss or Brier score could provide more refined assessment by considering probability distributions. Additionally, the absence of formal cross-validation in this research means absolute accuracy figures may be slightly optimistic, as models were evaluated on the same data used for training. However, since all models received identical treatment, relative performance comparisons remain valid.

The static nature of the models being trained on full-season data and applied uniformly, fail to account for within-season dynamics. Pitchers adjust approaches based on effectiveness, opponent adaptations, or physical changes such as fatigue or minor injury. A model trained on April-September data may poorly predict behavior if a pitcher substantially alters their strategy late in the season or during the playoffs.

5.4.3 Contextual Limitations

This research examines pitch selection in isolation from broader game context: score differential, base-out state, inning, bullpen availability, and defensive positioning. However, these all affect pitch selection but were excluded to preserve implementability. The analysis also treats all pitches as purely pitcher-driven, ignoring catchers' significant role in pitch calling. In modern MLB, catchers influence pitch selection substantially, particularly for younger pitchers who might

defer more heavily to catcher judgment, meaning models may miss important catcher-pitcher dynamic effects.

5.5 Future Research Directions

5.5.1 Multi-Season Temporal Analysis

Analyzing 3-5 seasons of data would enable assessment of temporal stability in pitcher patterns and investigation of how individual pitchers' predictability evolves over careers. Multi-season analysis could also address whether models trained on past seasons successfully predict future seasons, and whether career-stage patterns emerge. Do rookie pitchers show high predictability that decreases as they develop more sophisticated sequencing strategies?

5.5.2 Hierarchical and Bayesian Modeling

Rather than building completely separate models for each pitcher, hierarchical Bayesian approaches could allow pitchers to “borrow strength” from each other while preserving individual differences. Pitchers with similar characteristics like comparable arsenals, similar roles, comparable experience levels, likely share some strategic patterns. This approach would improve accuracy for pitchers with limited data while maintaining flexibility for well-characterized pitchers. Bayesian methods also measure uncertainty more rigorously, producing a range of possible outcomes that show how confident we can be in each prediction and whether that confidence is high enough to act on.

5.5.3 Alternative Modeling Frameworks

While multinomial logistic regression worked well, other approaches are worth exploring. Recurrent neural networks could better capture patterns across longer sequences of pitches than the two-pitch look back used here. Hidden Markov Models might uncover hidden strategic tendencies such as whether a pitcher is in an aggressive or cautious mode which could shape their pitch selection. Reinforcement learning could model pitchers as agents who gradually learn the best strategies through experience. Although early testing showed Random Forests and XGBoost offering no real improvement over logistic regression, more advanced combinations of multiple methods might still outperform any single model while also giving better estimates of prediction confidence.

5.5.4 Substantive Baseball Research Questions

Several substantive questions emerge from this work. Does pitcher predictability correlate with performance metrics like ERA, WHIP, or WAR? And if so, in which direction? Do hitters adjust their approach against predictable pitchers differently than against unpredictable ones, and do patient hitters exploit predictability more effectively than aggressive hitters? Could changes in predictability patterns serve as an early injury indicator, since a pitcher who suddenly becomes predictable may be favoring certain pitches due to discomfort? Finally, controlled field experiments in minor leagues could test whether predictive models actually improve offensive performance when deployed, moving beyond correlational evidence to causal evaluation of model value.

6. Conclusion

This research demonstrates that pitch sequences in Major League Baseball exhibit meaningful and exploitable predictability through multinomial logistic regression models incorporating lagged variables. The comprehensive model achieves 62.36% accuracy for pitch type prediction which is a substantial improvement over baseline expectations and a 15-percentage-point advantage over simpler models.

Four key insights emerge from this work. First, pitch selection is fundamentally driven by game state, particularly the count, rather than by simple sequential patterns alone. Second, pitcher predictability varies enormously across individuals and is strongly associated with pitcher role like relief pitchers with narrow repertoires who are systematically more predictable than starting pitchers who vary their approach across full outings. Third, model complexity must be carefully calibrated to available data, as demonstrated by the catastrophic failure of the interaction model when outcome categories expanded beyond what the data could support. Fourth, jointly predicting pitch type and location simultaneously introduces a compounding accuracy penalty, with the approximated combined accuracy of Model 4 (34.32%) reflecting the difficulty of requiring correctness on two dimensions at once.

These findings have practical implications for baseball operations. Teams could deploy models selectively by prioritizing relief pitchers and high-predictability opponents, use them diagnostically for pitcher self-assessment, or integrate predictions into pre-game scouting reports. However, implementation must account for accuracy ceilings leaving 35–40% of pitch type pre-

dictions incorrect even under the best model specification.

The methodological contribution extends beyond baseball. The successful integration of categorical lagged variables into multinomial logistic regression provides a framework applicable to diverse sequential categorical prediction problems in healthcare, customer analytics, education, and other domains where behavior unfolds in structured sequences.

Substantial opportunities remain for future research, including multi-season analysis to assess temporal stability, hierarchical Bayesian approaches to improve estimates for low-workload pitchers, and field experiments testing whether predictions improve offensive performance when actually deployed. As baseball continues integrating statistical modeling into strategy, approaches balancing predictive performance against practical implementability and interpretability will prove most valuable.

The question is not whether pitch sequences can be predicted, but rather how much prediction accuracy provides sufficient value to justify the analytical investment, cognitive load, and potential strategic adaptations that widespread deployment would entail. This research establishes that prediction is feasible and achieves non-trivial accuracy and that the most productive targets for deployment are relief pitchers whose defined roles and narrow repertoires make them systematically more transparent to statistical modeling.

7. R Shiny App Link

https://b-bruxvoort.shinyapps.io/pitch_type_prediction_app/

References

- Augustine, M. (2024). MLB pitch predictor: Using machine learning to previse pitch sequencing. *Medium*. <https://medium.com/@MichaelAugustine78/mlb-pitch-predictor-using-machine-learning-to-previse-pitch-sequencing-c5d38b9ac97c>
- Boddy, K. (2012). *Choosing the correct pitch sequences: Data-driven decisions*. Driveline Baseball. <https://www.drivelinebaseball.com/2012/05/choosing-the-correct-pitch-sequences-data-driven-decisions/>
- Clay, J. Z. (2023). Strategic pitch location: The role of two-pitch sequences in pitching success. *Baseball Research Journal*, 52(1).
- Ganeshapillai, G., & Gutttag, J. (2012). Predicting the next pitch. *Proceedings of the MIT Sloan Sports Analytics Conference*.
- Hastie, T., & Powers, A. D. (2011). *Predicting baseball outcomes using logistic regression*. https://hastie.su.domains/public/Papers/Powers_baseball.pdf
- Lee, J. S. (2022). Prediction of pitch type and location in baseball using ensemble model of deep neural networks. *Journal of Sports Analytics, Preprint*, 1–12. <https://doi.org/10.3233/JSA-200559>
- Major League Baseball. (2024). *Statcast search*. <https://baseballsavant.mlb.com>
- Stratton, C. (2018). *Predicting the PITCHf/x pitch classifier* [Master's Thesis]. Montana State University.

Appendix - R Code

7.1 Appendix A: Model Building Code

7.1.1 Model 1: Simple Model

```
build_simple_pitcher_model <- function(pitcher_data) {  
  tryCatch({  
    model <- multinom(pitch_type ~  
      prev_pitch_type_1 +  
      prev_pitch_type_2 +  
      pitch_number, data = pitcher_data,  
      maxit = 500, trace = FALSE)  
    return(model)  
  }, error = function(e) {  
    return(NULL)  
  })  
}  
  
pitcher_simple_models <- list()
```

```

for (pitcher in pitchers_sufficient_data) {

  cat("\nBuilding simple model for:",

      pitcher)

  pitcher_data <- pitch_data_simple %>%

    filter(player_name == pitcher)

  model <- build_simple_pitcher_model(pitcher_data)

  if (!is.null(model)) {

    pitcher_simple_models[[pitcher]] <- model

    cat(" - Success")

  } else {

    cat(" - Failed")

  }

}

cat("\n\nTotal simple models built:",

    length(pitcher_simple_models),

    "\n")

evaluate_simple_pitcher_model <- function(pitcher_name,

```

```

model, pitcher_data) {

tryCatch({

  predictions <- predict(model,

    newdata = pitcher_data,

    type = "class")

  accuracy <- mean(as.character(predictions) ==

    as.character(pitcher_data$pitch_type),

    na.rm = TRUE)

  conf_matrix <- tryCatch({

    pred_char <- as.character(predictions)

    actual_char <- as.character(pitcher_data$pitch_type)

    common_levels <- sort(unique(c(pred_char,

      actual_char)))

    confusionMatrix(factor(pred_char,

      levels = common_levels),

      factor(actual_char,

        levels = common_levels))

  }, error = function(e) {

```



```

        NULL

    })

    return(list(pitcher = pitcher_name,
               accuracy = accuracy,
               confusion_matrix = conf_matrix,
               error = NULL))
  }, error = function(e) {
    return(list(pitcher = pitcher_name,
               accuracy = NA, confusion_matrix = NULL,
               error = e$message))
  })
}

simple_model_evaluations <- list()

for (pitcher in names(pitcher_simple_models)) {
  pitcher_data <- pitch_data_simple %>%
    filter(player_name == pitcher)

  eval <- evaluate_simple_pitcher_model(pitcher,
    pitcher_simple_models[[pitcher]],

```

```

    pitcher_data)

    simple_model_evaluations[[pitcher]] <- eval
}

simple_accuracy_summary <- data.frame(pitcher = names(simple_model_evaluations),

  accuracy = sapply(simple_model_evaluations,

    function(x) x$accuracy),

  error = sapply(simple_model_evaluations,

    function(x) {

      if (!is.null(x$error))

        return(x$error) else return(NA)

    }))) %>%

  arrange(desc(accuracy))

print(simple_accuracy_summary)

cat("\n\nSimple Model Statistics:\n")

cat("Models with errors:", sum(!is.na(simple_accuracy_summary$error)),

  "\n")

cat("Models successful:", sum(is.na(simple_accuracy_summary$error) &

  !is.na(simple_accuracy_summary$accuracy)),

  "\n")

cat("Mean accuracy:", round(mean(simple_accuracy_summary$accuracy),

```

```

    na.rm = TRUE), 3), "\n")

cat("Median accuracy:", round(median(simple_accuracy_summary$accuracy,
    na.rm = TRUE), 3), "\n")

cat("Min accuracy:", round(min(simple_accuracy_summary$accuracy,
    na.rm = TRUE), 3), "\n")

cat("Max accuracy:", round(max(simple_accuracy_summary$accuracy,
    na.rm = TRUE), 3), "\n")

```

7.1.2 Model 2: Simple Model with Batter Handness (Stand)

```

build_simple_pitcher_model <- function(pitcher_data) {

  tryCatch({

    model <- multinom(pitch_type ~

      prev_pitch_type_1 +

      prev_pitch_type_2 +

      pitch_number +

      stand, data = pitcher_data,

      maxit = 500, trace = FALSE)

    return(model)

  }, error = function(e) {

    return(NULL)

  })

}

```

```

pitcher_simple_models <- list()

for (pitcher in pitchers_sufficient_data) {
  cat("\nBuilding simple model for:",
      pitcher)

  pitcher_data <- pitch_data_simple %>%
    filter(player_name == pitcher)

  model <- build_simple_pitcher_model(pitcher_data)

  if (!is.null(model)) {
    pitcher_simple_models[[pitcher]] <- model
    cat(" - Success")
  } else {
    cat(" - Failed")
  }
}

cat("\n\nTotal simple models built:",
    length(pitcher_simple_models),
    "\n")

```

```

evaluate_simple_pitcher_model <- function(pitcher_name,
  model, pitcher_data) {
  tryCatch({
    predictions <- predict(model,
      newdata = pitcher_data,
      type = "class")

    accuracy <- mean(as.character(predictions) ==
      as.character(pitcher_data$pitch_type),
      na.rm = TRUE)

    conf_matrix <- tryCatch({
      pred_char <- as.character(predictions)
      actual_char <- as.character(pitcher_data$pitch_type)

      common_levels <- sort(unique(c(pred_char,
        actual_char)))

      confusionMatrix(factor(pred_char,
        levels = common_levels),
        factor(actual_char,
          levels = common_levels))
    })
  })
}

```

```

    }, error = function(e) {

      NULL

    })

    return(list(pitcher = pitcher_name,

      accuracy = accuracy,

      confusion_matrix = conf_matrix,

      error = NULL))

  }, error = function(e) {

    return(list(pitcher = pitcher_name,

      accuracy = NA, confusion_matrix = NULL,

      error = e$message))

  })

}

simple_model_evaluations <- list()

for (pitcher in names(pitcher_simple_models)) {

  pitcher_data <- pitch_data_simple %>%

    filter(player_name == pitcher)

  eval <- evaluate_simple_pitcher_model(pitcher,

```

```

        pitcher_simple_models[[pitcher]],

        pitcher_data)

    simple_model_evaluations[[pitcher]] <- eval
}

simple_accuracy_summary <- data.frame(pitcher = names(simple_model_evaluations),

    accuracy = sapply(simple_model_evaluations,

        function(x) x$accuracy),

    error = sapply(simple_model_evaluations,

        function(x) {

            if (!is.null(x$error))

                return(x$error) else return(NA)

        }))) %>%

    arrange(desc(accuracy))

print(simple_accuracy_summary)

cat("\n\nSimple Model Statistics:\n")

cat("Models with errors:", sum(!is.na(simple_accuracy_summary$error)),

    "\n")

cat("Models successful:", sum(is.na(simple_accuracy_summary$error) &

    !is.na(simple_accuracy_summary$accuracy)),

```

```

"\n")

cat("Mean accuracy:", round(mean(simple_accuracy_summary$accuracy,
  na.rm = TRUE), 3), "\n")

cat("Median accuracy:", round(median(simple_accuracy_summary$accuracy,
  na.rm = TRUE), 3), "\n")

cat("Min accuracy:", round(min(simple_accuracy_summary$accuracy,
  na.rm = TRUE), 3), "\n")

cat("Max accuracy:", round(max(simple_accuracy_summary$accuracy,
  na.rm = TRUE), 3), "\n")

```

7.1.3 Model 3: Basic Model

```

build_pitcher_model <- function(pitcher_data) {

  tryCatch({

    model <- multinom(pitch_type ~

      prev_pitch_type_1 +

      prev_zone_1 + prev_pitch_type_2 +

      prev_zone_2 + balls +

      strikes + stand,

      data = pitcher_data,

      maxit = 500, trace = FALSE)

    return(model)

  }, error = function(e) {

```



```

        return(NULL)
    })
}

pitcher_models <- list()

for (pitcher in pitchers_sufficient_data) {
    cat("\nBuilding model for:",
        pitcher)

    pitcher_data <- pitch_data_features %>%
        filter(player_name == pitcher)

    model <- build_pitcher_model(pitcher_data)

    if (!is.null(model)) {
        pitcher_models[[pitcher]] <- model
        cat(" - Success")
    } else {
        cat(" - Failed")
    }
}

```

```

cat("\n\nTotal models built:",
    length(pitcher_models))

evaluate_pitcher_model <- function(pitcher_name,
    model, pitcher_data) {
  tryCatch({
    predictions <- predict(model,
        newdata = pitcher_data,
        type = "class")

    accuracy <- mean(as.character(predictions) ==
        as.character(pitcher_data$pitch_type),
        na.rm = TRUE)

    conf_matrix <- tryCatch({
      pred_char <- as.character(predictions)
      actual_char <- as.character(pitcher_data$pitch_type)

      common_levels <- sort(unique(c(pred_char,
          actual_char)))

      confusionMatrix(factor(pred_char,

```

```

        levels = common_levels),
        factor(actual_char,
               levels = common_levels))
    }, error = function(e) {
        NULL
    })

    return(list(pitcher = pitcher_name,
               accuracy = accuracy,
               confusion_matrix = conf_matrix,
               error = NULL))
  }, error = function(e) {
    return(list(pitcher = pitcher_name,
               accuracy = NA, confusion_matrix = NULL,
               error = e$message))
  })
}

model_evaluations <- list()

for (pitcher in names(pitcher_models)) {
  pitcher_data <- pitch_data_features %>%

```

```

    filter(player_name == pitcher)

eval <- evaluate_pitcher_model(pitcher,
    pitcher_models[[pitcher]],
    pitcher_data)
model_evaluations[[pitcher]] <- eval
}

accuracy_summary <- data.frame(pitcher = names(model_evaluations),
    accuracy = sapply(model_evaluations,
        function(x) x$accuracy),
    error = sapply(model_evaluations,
        function(x) {
            if (!is.null(x$error))
                return(x$error) else return(NA)
        }) %>%
    arrange(desc(accuracy))

print(accuracy_summary)

cat("\n\nModels with errors:",
    sum(!is.na(accuracy_summary$error)),

```

```

"\n")

cat("Models successful:", sum(is.na(accuracy_summary$error) &
  !is.na(accuracy_summary$accuracy)),
  "\n")

if (sum(!is.na(accuracy_summary$error)) >
  0) {
  cat("\n\nExample errors:\n")
  error_examples <- accuracy_summary %>%
    filter(!is.na(error)) %>%
    head(5)
  print(error_examples)
}

```

7.1.4 Model 4: Separate Type and Zone Models

```

build_pitcher_type_model <- function(pitcher_data) {
  tryCatch({
    model <- multinom(pitch_type ~
      prev_pitch_type_1 +
      prev_zone_1 + prev_pitch_type_2 +
      prev_zone_2 + balls +
      strikes + stand,

```

```

        data = pitcher_data,

        maxit = 500, trace = FALSE)

    return(model)

}, error = function(e) {

    return(NULL)

})

}

build_pitcher_zone_model <- function(pitcher_data) {

    tryCatch({

        model <- multinom(zone ~

            prev_pitch_type_1 +

            prev_zone_1 + prev_pitch_type_2 +

            prev_zone_2 + balls +

            strikes + stand,

            data = pitcher_data,

            maxit = 500, trace = FALSE)

        return(model)

    }, error = function(e) {

        return(NULL)

    })

}

```

```

pitcher_type_models <- list()

pitcher_zone_models <- list()

for (pitcher in pitchers_sufficient_data) {

  cat("\nBuilding models for:",

      pitcher)

  pitcher_data <- pitch_data_features %>%

    filter(player_name == pitcher)

  type_model <- build_pitcher_type_model(pitcher_data)

  if (!is.null(type_model)) {

    pitcher_type_models[[pitcher]] <- type_model

    cat(" - Type: Success")

  } else {

    cat(" - Type: Failed")

  }

  zone_model <- build_pitcher_zone_model(pitcher_data)

  if (!is.null(zone_model)) {

    pitcher_zone_models[[pitcher]] <- zone_model

    cat(" | Zone: Success")

```

```

    } else {

      cat(" | Zone: Failed")

    }
  }

cat("\n\nTotal pitch type models built:",

    length(pitcher_type_models))

cat("\nTotal zone models built:",

    length(pitcher_zone_models))

evaluate_pitcher_type_model <- function(pitcher_name,

  model, pitcher_data) {

  tryCatch({

    predictions <- predict(model,

      newdata = pitcher_data,

      type = "class")

    accuracy <- mean(as.character(predictions) ==

      as.character(pitcher_data$pitch_type),

      na.rm = TRUE)

    conf_matrix <- tryCatch({

      pred_char <- as.character(predictions)

```



```

    actual_char <- as.character(pitcher_data$pitch_type)

    common_levels <- sort(unique(c(pred_char,
                                    actual_char)))

    confusionMatrix(factor(pred_char,
                            levels = common_levels),
                    factor(actual_char,
                            levels = common_levels))

  }, error = function(e) NULL)

return(list(pitcher = pitcher_name,
            accuracy = accuracy,
            confusion_matrix = conf_matrix,
            error = NULL))

}, error = function(e) {

  return(list(pitcher = pitcher_name,
              accuracy = NA, confusion_matrix = NULL,
              error = e$message))

})

}

evaluate_pitcher_zone_model <- function(pitcher_name,

```

```

model, pitcher_data) {

  tryCatch({

    predictions <- predict(model,

      newdata = pitcher_data,

      type = "class")

    accuracy <- mean(as.character(predictions) ==

      as.character(pitcher_data$zone),

      na.rm = TRUE)

    conf_matrix <- tryCatch({

      pred_char <- as.character(predictions)

      actual_char <- as.character(pitcher_data$zone)

      common_levels <- sort(unique(c(pred_char,

        actual_char)))

      confusionMatrix(factor(pred_char,

        levels = common_levels),

        factor(actual_char,

          levels = common_levels))

    }, error = function(e) NULL)

    return(list(pitcher = pitcher_name,

```

```

        accuracy = accuracy,

        confusion_matrix = conf_matrix,

        error = NULL))

}, error = function(e) {

    return(list(pitcher = pitcher_name,

        accuracy = NA, confusion_matrix = NULL,

        error = e$message))

})

}

type_model_evaluations <- list()

for (pitcher in names(pitcher_type_models)) {

    pitcher_data <- pitch_data_features %>%

        filter(player_name == pitcher)

    eval <- evaluate_pitcher_type_model(pitcher,

        pitcher_type_models[[pitcher]],

        pitcher_data)

    type_model_evaluations[[pitcher]] <- eval

}

zone_model_evaluations <- list()

```

```

for (pitcher in names(pitcher_zone_models)) {
  pitcher_data <- pitch_data_features %>%
    filter(player_name == pitcher)

  eval <- evaluate_pitcher_zone_model(pitcher,
    pitcher_zone_models[[pitcher]],
    pitcher_data)

  zone_model_evaluations[[pitcher]] <- eval
}

type_accuracy_summary <- data.frame(pitcher = names(type_model_evaluations),
  type_accuracy = sapply(type_model_evaluations,
    function(x) x$accuracy),
  type_error = sapply(type_model_evaluations,
    function(x) {
      if (!is.null(x$error))
        return(x$error) else return(NA)
    })))

zone_accuracy_summary <- data.frame(pitcher = names(zone_model_evaluations),
  zone_accuracy = sapply(zone_model_evaluations,
    function(x) x$accuracy),

```

```

zone_error = sapply(zone_model_evaluations,

  function(x) {

    if (!is.null(x$error))

      return(x$error) else return(NA)

  })

combined_accuracy_summary <- type_accuracy_summary %>%

  full_join(zone_accuracy_summary,

    by = "pitcher") %>%

  arrange(desc(type_accuracy))

print(combined_accuracy_summary)

cat("\n\n=== PITCH TYPE MODEL PERFORMANCE ===\n")

cat("Models with errors:", sum(!is.na(type_accuracy_summary$type_error)),

  "\n")

cat("Models successful:", sum(is.na(type_accuracy_summary$type_error) &

  !is.na(type_accuracy_summary$type_accuracy)),

  "\n")

cat("Mean accuracy:", round(mean(type_accuracy_summary$type_accuracy,

  na.rm = TRUE), 4), "\n")

cat("\n\n=== ZONE MODEL PERFORMANCE ===\n")

```

```

cat("Models with errors:", sum(!is.na(zone_accuracy_summary$zone_error)),
    "\n")

cat("Models successful:", sum(is.na(zone_accuracy_summary$zone_error) &
    !is.na(zone_accuracy_summary$zone_accuracy)),
    "\n")

cat("Mean accuracy:", round(mean(zone_accuracy_summary$zone_accuracy,
    na.rm = TRUE), 4), "\n")

```

7.1.5 Model 5: Interaction Model

```

build_pitcher_interaction_model <- function(pitcher_data) {
  tryCatch({
    n_combinations <- n_distinct(pitcher_data$pitch_type_zone)
    n_pitches <- nrow(pitcher_data)

    cat(" | Combinations:",
        n_combinations, "| Pitches:",
        n_pitches)

    if (n_pitches/n_combinations <
        5) {
      cat(" - Too sparse")
      return(NULL)
    }
  }, error = function(e) {
    return(NULL)
  })
}

```

```

}

combination_counts <- pitcher_data %>%

  count(pitch_type_zone) %>%

  filter(n >= 3)

pitcher_data_filtered <- pitcher_data %>%

  filter(pitch_type_zone %in%

    combination_counts$pitch_type_zone) %>%

  mutate(pitch_type_zone = droplevels(pitch_type_zone))

cat(" | Filtered to:",

  n_distinct(pitcher_data_filtered$pitch_type_zone),

  "combinations")

if (nrow(pitcher_data_filtered) <

  50) {

  cat(" - Insufficient data after filtering")

  return(NULL)

}

model <- multinom(pitch_type_zone ~

```

```

    prev_pitch_type_1 +

    prev_zone_1 + balls +

    strikes + stand,

    data = pitcher_data_filtered,

    maxit = 1000, trace = FALSE,

    MaxNWts = 5000)

  return(model)
}, error = function(e) {

  cat(" - Error:", e$message)

  return(NULL)

})
}

pitcher_interaction_models <- list()

for (pitcher in pitchers_sufficient_data) {

  cat("\nBuilding interaction model for:",

    pitcher)

  pitcher_data <- pitch_data_features %>%

    filter(player_name == pitcher)

```



```

model <- build_pitcher_interaction_model(pitcher_data)

if (!is.null(model)) {

  pitcher_interaction_models[[pitcher]] <- model

  cat(" - Success")

} else {

  cat(" - Failed")

}
}

cat("\n\nTotal interaction models built:",

    length(pitcher_interaction_models))

evaluate_pitcher_interaction_model <- function(pitcher_name,

  model, pitcher_data) {

  tryCatch({

    predictions <- predict(model,

      newdata = pitcher_data,

      type = "class")

    accuracy <- mean(as.character(predictions) ==

      as.character(pitcher_data$pitch_type_zone),

```

```

na.rm = TRUE)

pred_split <- str_split_fixed(as.character(predictions),
  "_", 2)
actual_split <- str_split_fixed(as.character(pitcher_data$pitch_type_zone),
  "_", 2)

type_accuracy <- mean(pred_split[,
  1] == actual_split[,
  1], na.rm = TRUE)
zone_accuracy <- mean(pred_split[,
  2] == actual_split[,
  2], na.rm = TRUE)

conf_matrix <- tryCatch({
  pred_char <- as.character(predictions)
  actual_char <- as.character(pitcher_data$pitch_type_zone)
  common_levels <- sort(unique(c(pred_char,
    actual_char)))

  confusionMatrix(factor(pred_char,
    levels = common_levels),

```

```

        factor(actual_char,
               levels = common_levels))

    }, error = function(e) NULL)

return(list(pitcher = pitcher_name,
           combined_accuracy = accuracy,
           type_accuracy = type_accuracy,
           zone_accuracy = zone_accuracy,
           confusion_matrix = conf_matrix,
           error = NULL))

}, error = function(e) {

    return(list(pitcher = pitcher_name,
               combined_accuracy = NA,
               type_accuracy = NA,
               zone_accuracy = NA,
               confusion_matrix = NULL,
               error = e$message))

})

}

interaction_model_evaluations <- list()

```

```

for (pitcher in names(pitcher_interaction_models)) {

  pitcher_data <- pitch_data_features %>%

    filter(player_name == pitcher)

  eval <- evaluate_pitcher_interaction_model(pitcher,

    pitcher_interaction_models[[pitcher]],

    pitcher_data)

  interaction_model_evaluations[[pitcher]] <- eval
}

interaction_accuracy_summary <- data.frame(pitcher = names(interaction_model_evaluations),

  combined_accuracy = sapply(interaction_model_evaluations,

    function(x) x$combined_accuracy),

  type_accuracy = sapply(interaction_model_evaluations,

    function(x) x$type_accuracy),

  zone_accuracy = sapply(interaction_model_evaluations,

    function(x) x$zone_accuracy),

  error = sapply(interaction_model_evaluations,

    function(x) {

      if (!is.null(x$error))

        return(x$error) else return(NA)

    }) %>%

```

```

    arrange(desc(combined_accuracy))

print(interaction_accuracy_summary)

cat("\n\n=== INTERACTION MODEL PERFORMANCE ===\n")

cat("Models with errors:", sum(!is.na(interaction_accuracy_summary$error)),
    "\n")

cat("Models successful:", sum(is.na(interaction_accuracy_summary$error) &
    !is.na(interaction_accuracy_summary$combined_accuracy)),
    "\n")

cat("Mean combined accuracy:",
    round(mean(interaction_accuracy_summary$combined_accuracy,
        na.rm = TRUE), 4), "\n")

cat("Mean type accuracy (from interaction):",
    round(mean(interaction_accuracy_summary$type_accuracy,
        na.rm = TRUE), 4), "\n")

cat("Mean zone accuracy (from interaction):",
    round(mean(interaction_accuracy_summary$zone_accuracy,
        na.rm = TRUE), 4), "\n")

```